

## Data Science Academy - Curso de Pós-Graduação

### Pós-Graduação em Engenharia de Machine Learning (PGEML)

**Problema de Negócio:** Prever se uma determinada pessoa, dadas as suas características, recebeu, ou não, o Auxílio Emergencial do Governo Federal (BR) durante a pandemia do novo coronavírus (COVID-19).

#### 1) Instalando/ Importando os pacotes necessários:

#### 2) Carregando os dados em um Dataset/ Dataframe:

#### 3) Visualizando a estrutura da tabela:

#### 4) Pré-processamento (transformando/ limpando):

##### 4.1) ESCOLHENDO AS COLUNAS COM AS QUAIS IREMOS TRABALHAR:

##### 4.2) RENOMEANDO OS NOMES DE ALGUMAS COLUNAS:

- Renomeando a variável dependente **AUXILIO\_EMERGENCIAL (D0051)**:

- Renomeando mais algumas colunas:

##### 4.3) VERIFICAÇÃO DE DADOS MISSING E TRATANDO-OS/ REMOVENDO-OS:

##### 4.4) ""DE-PARA"" P/ ALGUMAS COLUNAS E EXPLORANDO-AS:

- Convertendo a variável **IDADE (A002)** para o tipo "category" (qualitativa ordinal) e fazendo o de-para:

- Convertendo a variável **SEXO (A003)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **COR (A004)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **ESCOLARIDADE (A005)** para o tipo "category" (qualitativa ordinal) e fazendo o de-para:

- Convertendo a variável **ESTAVA\_TRABALHANDO (C001)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **TIPO\_EMPREGO (C007)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **CARTEIRA\_ASSINADA (C007B)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **SOMENTE\_BENEFICIO (C0103)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **NAO\_REMUNERADO (C0104)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

- Convertendo a variável **UF** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

#### 4.5) MUDANDO O TIPO DE ALGUMAS VARIÁVEIS:

- Deixando as variáveis de dinheiro como tipo "float64" (Boa Prática) e as demais como "Category" ou "int64", conforme o caso:

- Convertendo a variável dependente **"TAG" (OU RÓTULO)** para o tipo "int32", apenas para deixarmos ela de fora do One-Hot Encoding posteriormente:

#### 4.6) ANONIMIZAÇÃO: (N/A)

OS DADOS QUE O IBGE DISPONIBILIZA JÁ SÃO ANONIMIZADOS!

#### 4.7) VERIFICAÇÃO DE DADOS MISSING E TRATANDO-OS/ REMOVENDO-OS:

### 5) Análise Exploratória/ Resumo Estatístico:

#### 5.1) GRÁFICO: "AUXÍLIO EMERGENCIAL P/ UNIDADE DA FEDERAÇÃO"

- Query (Tabela):

- Plot:

#### 5.2) GRÁFICO: "AUXÍLIO EMERGENCIAL P/ SEXO E COR":

- Query (Tabela):

- Plot:

#### 5.3) GRÁFICO: "AUXÍLIO EMERGENCIAL P/ FAIXA SALARIAL"

- Criando uma nova variável no final do dataframe 'df' p/ segmentar as faixas salariais:

- Query (Tabela):

- Plot:

- Excluindo a variável c/ as faixas salariais e tb a variável contadora 'COUNT' p/ não interferir no modelo preditivo:

#### 5.4) VERIFICANDO SE A CLASSE PREDITORA ESTÁ DESBALANCEADA:

- Verificando a proporção dos que RECEBEM o Auxílio Emergencial e dos que NÃO RECEBEM:

#### 5.5) OUTROS GRÁFICOS P/ ANÁLISE DAS DISTRIBUIÇÕES E OUTLIERS:

- Automatizando a criação de Boxplots (variáveis numéricas) e Gráf de Barras (categóricas):

## 6) Divisão Treino-Teste:

## 7) Pré-processamento/Padronização (Preparando p/ o algoritmo):

- Convertendo as variáveis categóricas em numéricas e usando o One-Hot Encoding:

Toda e qualquer transformação aplicada aos dados de treino, deve ser aplicada aos dados de teste e aos novos dados.

- Padronizando os dados para que estejam na mesma escala, usando o StandardScaler:

( Não é necessário no RandomForest, mas faremos para aprender )

Toda e qualquer transformação aplicada aos dados de treino, deve ser aplicada aos dados de teste e aos novos dados (nestes 2 últimos, sem o `*.fit()`)

## 8) Instanciando/ Criando o modelo:

## 9) Treinando o modelo:

## 10) Avaliando/ Visualizando o modelo:

- Gráfico da "Taxa de Erro" da Floresta Aleatória:

- Gráfico da "Importância de Cada Variável":

- Fazendo previsões com os dados de TESTE:

- Criando a Matriz de Confusão:

- Avaliação das métricas do modelo:

- Validação Cruzada para checar a performance em cada fold:

## 11) Otimização de Hiperparâmetros e Versão Final do Modelo:

- Otimização de Hiperparâmetros:

- Versão Final do Modelo:

## ->SALVAMENTO/ DEPLOY DO MODELO:

## ->CARREGANDO E TESTANDO O MODELO/PADRONIZADOR SALVOS:

```
In [139]: %watermark -a "Christopherson B. Francischetti"
```

Author: Christopherson B. Francischetti

# Início, 2

## Data Science Academy - Curso de Pós-Graduação

### Pós-Graduação em Engenharia de Machine Learning (PGEML)

**Problema de Negócio:** Prever se uma determinada pessoa, dadas as suas características, recebeu, ou não, o Auxílio Emergencial do Governo Federal (BR) durante a pandemia do novo coronavírus (COVID-19).

#### 1) Instalando/ Importando os pacotes necessários:

```
In [1]: # Imports
import joblib
import sklearn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix, ConfusionMatrixDisplay

import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: %reload_ext watermark
%watermark -a "Christopherson B. Francischetti"
```

Author: Christopherson B. Francischetti

#### 2) Carregando os dados em um Dataset/ Dataframe:

```
In [3]: # Carregando o dataframe

df_pnadcovid = pd.read_csv("PNAD_COVID_062020.csv")
print(df_pnadcovid.shape)
df_pnadcovid.head()
```

(381270, 114)

```
Out[3]:
```

|   | Ano  | UF | CAPITAL | RM_RIDE | V1008 | V1012 | V1013 | V1016 | Estrato | UPA       | ... | D0053  | D0061 | D0063 | D0071 | D0073 | F006 |
|---|------|----|---------|---------|-------|-------|-------|-------|---------|-----------|-----|--------|-------|-------|-------|-------|------|
| 0 | 2020 | 11 | 11.0    | NaN     | 1     | 4     | 6     | 2     | 1110011 | 110015970 | ... | 1200.0 | 2     | NaN   | 2     | NaN   |      |
| 1 | 2020 | 11 | 11.0    | NaN     | 1     | 4     | 6     | 2     | 1110011 | 110015970 | ... | 1200.0 | 2     | NaN   | 2     | NaN   |      |
| 2 | 2020 | 11 | 11.0    | NaN     | 1     | 4     | 6     | 2     | 1110011 | 110015970 | ... | 1200.0 | 2     | NaN   | 2     | NaN   |      |
| 3 | 2020 | 11 | 11.0    | NaN     | 1     | 4     | 6     | 2     | 1110011 | 110015970 | ... | 1200.0 | 2     | NaN   | 2     | NaN   |      |
| 4 | 2020 | 11 | 11.0    | NaN     | 2     | 1     | 6     | 2     | 1110011 | 110015970 | ... | NaN    | 2     | NaN   | 2     | NaN   |      |

5 rows × 114 columns

#### 3) Visualizando a estrutura da tabela:

```
In [4]: df_pnadcovid.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381270 entries, 0 to 381269
Columns: 114 entries, Ano to F006
dtypes: float64(71), int64(43)
memory usage: 331.6 MB
```

#### 4) Pré-processamento (transformando/ limpando):

#### 4.1) ESCOLHENDO AS COLUNAS COM AS QUAIS IREMOS TRABALHAR:

```
# Vamos trabalhar somente com algumas variáveis
df = df_pnadscovid[["UF", "V1022", "V1023", "A002", "A003", "A004", "A005", "C001", "C007", "C007B", "C007C", "C007D", "C010", "C01012", "C01022", "C0103", "C0104", "C014", "D0013", "D0023", "D0033", "D0043", "D0051", "D0063", "D0073", "COUNT"]]

# Criando ua Variável Contadora "COUNT"
df['COUNT'] = 1
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381270 entries, 0 to 381269
Data columns (total 26 columns):
#   Column      Non-Null Count  Dtype
---  -
0    UF           381270 non-null  int64
1    V1022        381270 non-null  int64
2    V1023        381270 non-null  int64
3    A002         381270 non-null  int64
4    A003         381270 non-null  int64
5    A004         381270 non-null  int64
6    A005         381270 non-null  int64
7    C001         314101 non-null  float64
8    C007         150551 non-null  float64
9    C007B        91110 non-null   float64
10   C007C        144816 non-null  float64
11   C007D        144816 non-null  float64
12   C010         146111 non-null  float64
13   C01012       141677 non-null  float64
14   C01022       1747 non-null    float64
15   C0103        84 non-null      float64
16   C0104        3134 non-null    float64
17   C014         245529 non-null  float64
18   D0013        130772 non-null  float64
19   D0023        19258 non-null   float64
20   D0033        33349 non-null   float64
21   D0043        12777 non-null   float64
22   D0051        381270 non-null  int64
23   D0063        8142 non-null    float64
24   D0073        18417 non-null   float64
25   COUNT        381270 non-null  int64

dtypes: float64(17), int64(9)
memory usage: 75.6 MB
```

## 4.2) RENOMEANDO OS NOMES DE ALGUMAS COLUNAS:

- Renomeando a variável dependente **AUXILIO\_EMERGENCIAL (D0051)**:

```
df.rename(columns = {'D0051': 'AUXILIO_EMERGENCIAL'}, inplace = True)
```

- **Renomeando mais algumas colunas:**

```
In [7]: df.rename(columns={'A002': 'IDADE'}, inplace=True)
df.rename(columns={'A003': 'SEXO'}, inplace=True)
df.rename(columns={'A004': 'COR'}, inplace=True)
df.rename(columns={'A005': 'ESCOLARIDADE'}, inplace=True)
df.rename(columns={'C001': 'ESTAVA_TRABALHANDO'}, inplace = True)
df.rename(columns={'C007': 'TIPO_EMPREGO'}, inplace=True)
df.rename(columns={'C007B': 'CARTEIRA_ASSINADA'}, inplace=True)
df.rename(columns={'C01012': 'SALARIO'}, inplace=True)
df.rename(columns={'C01022': 'VALOR_TICKET-CESTA'}, inplace=True)
df.rename(columns={'C0103': 'SOMENTE_BENEFICIO'}, inplace=True)
df.rename(columns={'C0104': 'NAO_REMUNERADO'}, inplace=True)
df.rename(columns={'D0013': 'SOMA_APOSENTADORIAS+PENSAO'}, inplace=True)
df.rename(columns={'D0033': 'SOMA_BOLSAS_FAMILIA'}, inplace=True)
df.rename(columns={'D0043': 'SOMA_BPC-LOAS'}, inplace=True)
df.rename(columns={'D0063': 'SOMA_SEGUROS_DESEMPREGO'}, inplace=True)
```

#### 4.3) VERIFICAÇÃO DE DADOS MISSING E TRATANDO-OS/ REMOVENDO-OS:

```
In [8]: df.isna().sum()
```

```
Out[8]: UF                                0
V1022                                     0
V1023                                     0
IDADE                                    0
SEXO                                     0
COR                                      0
ESCOLARIDADE                            0
ESTAVA_TRABALHANDO                      67169
TIPO_EMPREGO                            230719
CARTEIRA_ASSINADA                       290160
C007C                                    236454
C007D                                    236454
C010                                     235159
SALARIO                                 239593
VALOR_TICKET-CESTA                      379523
SOMENTE_BENEFICIO                       381186
NAO_REMUNERADO                          378136
C014                                    135741
SOMA_APOSENTADORIAS+PENSAO              250498
D0023                                    362012
SOMA_BOLSAS_FAMILIA                    347921
SOMA_BPC-LOAS                          368493
AUXILIO_EMERGENCIAL                     0
SOMA_SEGUROS_DESEMPREGO                 373128
D0073                                    362853
COUNT                                  0
dtype: int64
```

```
In [9]: # Substituindo valores ausentes (imputação)
df['ESTAVA_TRABALHANDO'].fillna(99, inplace = True)
df['TIPO_EMPREGO'].fillna(99, inplace = True)
df['CARTEIRA_ASSINADA'].fillna(99, inplace = True)
df['C007C'].fillna(99, inplace = True)
df['C007D'].fillna(99, inplace = True)
df['C010'].fillna(99, inplace = True)
df['SALARIO'].fillna(0, inplace = True)
df['VALOR_TICKET-CESTA'].fillna(0, inplace = True)
df['SOMENTE_BENEFICIO'].fillna(99, inplace = True)
df['NAO_REMUNERADO'].fillna(99, inplace = True)
df['C014'].fillna(99, inplace = True)
df['SOMA_APOSENTADORIAS+PENSAO'].fillna(0, inplace = True)
df['D0023'].fillna(0, inplace = True)
df['SOMA_BOLSAS_FAMILIA'].fillna(0, inplace = True)
df['SOMA_BPC-LOAS'].fillna(0, inplace = True)
df['SOMA_SEGUROS_DESEMPREGO'].fillna(0, inplace = True)
df['D0073'].fillna(0, inplace = True)
```

```
In [10]: print(df.shape)
df.isna().sum()
```

```
(381270, 26)
```

```
Out[10]: UF                                0
V1022                                       0
V1023                                       0
IDADE                                       0
SEXO                                         0
COR                                           0
ESCOLARIDADE                             0
ESTAVA_TRABALHANDO                         0
TIPO_EMPREGO                               0
CARTEIRA_ASSINADA                          0
C007C                                       0
C007D                                       0
C010                                         0
SALARIO                                     0
VALOR_TICKET-CESTA                         0
SOMENTE_BENEFICIO                          0
NAO_REMUNERADO                             0
C014                                         0
SOMA_APOSENTADORIAS+PENSAO                 0
D0023                                       0
SOMA_BOLSAS_FAMILIA                        0
SOMA_BPC-LOAS                              0
AUXILIO_EMERGENCIAL                        0
SOMA_SEGUROS_DESEMPREGO                    0
D0073                                       0
COUNT                                     0
dtype: int64
```

#### 4.4) "DE-PARA" P/ ALGUMAS COLUNAS E EXPLORANDO-AS:

- Convertendo a variável **IDADE (A002)** para o tipo "category" (qualitativa ordinal) e fazendo o de-para:

```
In [11]: # Vendo as primeiras linhas da coluna IDADE
df['IDADE'].head()
```

```
Out[11]: 0    35
1     29
2     13
3     10
4     57
Name: IDADE, dtype: int64
```

```
In [12]: # Valor Mínimo e Valor Máximo
# df.IDADE.describe()

print(df['IDADE'].min())
print(df['IDADE'].max())

0
111
```

```
In [13]: # Convertendo a variável IDADE para categórica
# onde de 0 a 4 = "-5", de 5 a 11 = "5-11", de 12 a 17 = "12-17", de 18 a 49 = "18-49", de 50 a 64 = "50-64", de 65+ = "65+"

df['IDADE'] = pd.cut(df['IDADE'], \
    bins=[-1, 4, 11, 17, 49, 64, 111], labels=['-5', '5-11', '12-17', '18-49', '50-64', '65+'])
df['IDADE'].head()

# OBS: Precisei iniciar o bins em "-1" pois, por algum motivo, não estava sendo incluído a idade "0"
```

```
Out[13]: 0    18-49
1    18-49
2    12-17
3     5-11
4    50-64
Name: IDADE, dtype: category
Categories (6, object): ['-5' < '5-11' < '12-17' < '18-49' < '50-64' < '65+']
```

• Convertendo a variável SEXO (A003) para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [14]: # Vendo as primeiras linhas da coluna SEXO

df['SEXO'].head() #Outra forma: df.SEXO.head()
```

```
Out[14]: 0    1
1    2
2    1
3    1
4    2
Name: SEXO, dtype: int64
```

```
In [15]: # Valor Mínimo e Valor Máximo

# df.SEXO.describe()

print(df['SEXO'].min())
print(df['SEXO'].max())
```

```
1
2
```

```
In [16]: # Convertendo a variável SEXO para categórica onde 1 = Homem e 2 = Mulher

df['SEXO'] = df['SEXO'].astype('category')
df['SEXO'] = df['SEXO'].cat.rename_categories(['Homem', 'Mulher'])
df['SEXO'].head()
```

```
Out[16]: 0    Homem
1    Mulher
2    Homem
3    Homem
4    Mulher
Name: SEXO, dtype: category
Categories (2, object): ['Homem', 'Mulher']
```

• Convertendo a variável COR (A004) para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [17]: # Vendo as primeiras linhas da coluna COR

df.COR.head() #ou df['COR'].head() , caso o nome da col tenha espaços (seria obrigado a fazer assim)
```

```
Out[17]: 0    4
1    4
2    4
3    4
4    1
Name: COR, dtype: int64
```

In [18]: *# Valor Mínimo e Valor Máximo*

```
# df.COR.describe()

print(df['COR'].min())
print(df['COR'].max())
```

1  
9

In [19]: *# Descobrindo os valores únicos da variável COR*

```
ValoresOrdenados = df['COR'].unique()
ValoresOrdenados.sort()
ValoresOrdenados
```

Out[19]: array([1, 2, 3, 4, 5, 9], dtype=int64)

In [20]: *# Contando os valores de cada estrato*

```
print(df['COR'].value_counts()[1])
print(df['COR'].value_counts()[2])
print(df['COR'].value_counts()[3])
print(df['COR'].value_counts()[4])
print(df['COR'].value_counts()[5])
print(df['COR'].value_counts()[9])
```

159887  
31309  
2349  
186140  
1487  
98

In [21]: *# Mostrando o resultado anterior em termos percentuais*

```
print(df['COR'].value_counts(normalize=True)[1])
print(df['COR'].value_counts(normalize=True)[2])
print(df['COR'].value_counts(normalize=True)[3])
print(df['COR'].value_counts(normalize=True)[4])
print(df['COR'].value_counts(normalize=True)[5])
print(df['COR'].value_counts(normalize=True)[9])
```

0.4193537388202586  
0.08211765940147402  
0.0061609882760248645  
0.4882104545335327  
0.003900123272221785  
0.00025703569648805307

In [22]: *# Convertendo a variável COR para categórica (qualitativa nominal)*

```
df['COR'] = df['COR'].astype('category')
df['COR'] = df['COR'].cat.rename_categories(['Branca', 'Preta', 'Amarela', 'Parda', 'Indígena', 'Ignorado'])
df['COR'].head()
```

Out[22]: 0 Parda  
1 Parda  
2 Parda  
3 Parda  
4 Branca  
Name: COR, dtype: category  
Categories (6, object): ['Branca', 'Preta', 'Amarela', 'Parda', 'Indígena', 'Ignorado']

• Convertendo a variável **ESCOLARIDADE (A005)** para o tipo "category" (qualitativa ordinal) e fazendo o de-para:

```
In [23]: # Vendo as primeiras linhas da coluna ESCOLARIDADE
```

```
df['ESCOLARIDADE'].head()
```

```
Out[23]: 0    5
         1    7
         2    2
         3    2
         4    2
         Name: ESCOLARIDADE, dtype: int64
```

```
In [24]: # Valor Mínimo e Valor Máximo
```

```
# df.ESCOLARIDADE.describe()

print(df['ESCOLARIDADE'].min())
print(df['ESCOLARIDADE'].max())
```

```
1
8
```

```
In [25]: # Convertendo a variável ESCOLARIDADE para categórica, onde:
```

```
#
# de 1 a 2 = "Sem instrução ou Fundamental Incompleto",
# de 3 a 4 = "Fundamental Completo ou Médio Incompleto",
# de 5 a 6 = "Médio Completo ou Superior Incompleto",
# 7 = "Superior Completo",
# 8 = "Pós-graduação"

df['ESCOLARIDADE'] = pd.cut(df['ESCOLARIDADE'], \
    bins=[0, 2, 4, 6, 7, 8], \
    labels=['Sem instrução ou Fundamental Incompleto', \
            'Fundamental Completo ou Médio Incompleto', \
            'Médio Completo ou Superior Incompleto', \
            'Superior Completo', \
            'Pós-graduação'])

df['ESCOLARIDADE'].head()
```

```
Out[25]: 0    Médio Completo ou Superior Incompleto
         1    Superior Completo
         2    Sem instrução ou Fundamental Incompleto
         3    Sem instrução ou Fundamental Incompleto
         4    Sem instrução ou Fundamental Incompleto
         Name: ESCOLARIDADE, dtype: category
         Categories (5, object): ['Sem instrução ou Fundamental Incompleto' < 'Fundamental Completo ou Médio Inco
         mpleto' < 'Médio Completo ou Superior Incompleto' < 'Superior Completo' < 'Pós-graduação']
```

• Convertendo a variável **ESTAVA\_TRABALHANDO (C001)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [26]: # Vendo as primeiras linhas da coluna ESTAVA_TRABALHANDO
```

```
df['ESTAVA_TRABALHANDO'].head() #Outra forma: df.ESTAVA_TRABALHANDO.head()
```

```
Out[26]: 0    1.0
         1    2.0
         2   99.0
         3   99.0
         4    2.0
         Name: ESTAVA_TRABALHANDO, dtype: float64
```

```
In [27]: # Valor Mínimo e Valor Máximo
```

```
# df.ESTAVA_TRABALHANDO.describe()

print(df['ESTAVA_TRABALHANDO'].min())
print(df['ESTAVA_TRABALHANDO'].max())
```

```
1.0
99.0
```

```
In [28]: # Descobrindo os valores únicos da variável ESTAVA_TRABALHANDO
```

```
ValoresOrdenados = df['ESTAVA_TRABALHANDO'].unique()
ValoresOrdenados.sort()
ValoresOrdenados
```

```
Out[28]: array([ 1.,  2., 99.])
```

```
In [29]: # Convertendo a variável ESTAVA_TRABALHANDO para categórica onde 1=Sim; 2=Não e 99=N/A
```

```
df['ESTAVA_TRABALHANDO'] = df['ESTAVA_TRABALHANDO'].astype('category')
df['ESTAVA_TRABALHANDO'] = df['ESTAVA_TRABALHANDO'].cat.rename_categories(['Sim', 'Não', 'N/A'])
df['ESTAVA_TRABALHANDO'].head()
```

```
Out[29]: 0    Sim
         1    Não
         2    N/A
         3    N/A
         4    Não
         Name: ESTAVA_TRABALHANDO, dtype: category
         Categories (3, object): ['Sim', 'Não', 'N/A']
```

• Convertendo a variável TIPO\_EMPREGO (C007) para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [30]: # Vendo as primeiras Linhas da coluna TIPO_EMPREGO
```

```
df.TIPO_EMPREGO.head() # ou df['TIPO_EMPREGO'].head()
```

```
Out[30]: 0    4.0
         1    7.0
         2   99.0
         3   99.0
         4   99.0
         Name: TIPO_EMPREGO, dtype: float64
```

```
In [31]: # Valor Mínimo e Valor Máximo
```

```
# df.TIPO_EMPREGO.describe()

print(df['TIPO_EMPREGO'].min())
print(df['TIPO_EMPREGO'].max())
```

```
1.0
99.0
```

```
In [32]: # Descobrindo os valores únicos da variável TIPO_EMPREGO
```

```
ValoresOrdenados = df['TIPO_EMPREGO'].unique()
ValoresOrdenados.sort()
ValoresOrdenados
```

```
Out[32]: array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 99.])
```

In [33]: *# Contando os valores de cada estrato*

```
print(df['TIPO_EMPREGO'].value_counts()[1])
print(df['TIPO_EMPREGO'].value_counts()[2])
print(df['TIPO_EMPREGO'].value_counts()[3])
print(df['TIPO_EMPREGO'].value_counts()[4])
print(df['TIPO_EMPREGO'].value_counts()[5])
print(df['TIPO_EMPREGO'].value_counts()[6])
print(df['TIPO_EMPREGO'].value_counts()[7])
print(df['TIPO_EMPREGO'].value_counts()[8])
print(df['TIPO_EMPREGO'].value_counts()[9])
print(df['TIPO_EMPREGO'].value_counts()[99])
```

```
7804
617
678
61435
21871
4616
45905
3185
4440
230719
```

In [34]: *# Convertendo a variável TIPO\_EMPREGO para categórica, onde:*

```
# 1 = "Trabalhador doméstico (empregado doméstico, cuidados, babá)",
# 2 = "Militar",
# 3 = "Policia! militar ou bombeiro militar",
# 4 = "Empregado do setor privado",
# 5 = "Empregado do setor público"
# 6 = "Empregador"
# 7 = "Autônomo (Conta própria)"
# 8 = "Trabalhador familiar não remunerado em ajuda a parente"
# 9 = "Estava fora do mercado de trabalho"
# 99 = "N/A"

df['TIPO_EMPREGO'] = df['TIPO_EMPREGO'].astype('category')
df['TIPO_EMPREGO'] = df['TIPO_EMPREGO'].cat.rename_categories(\
    ['Trabalhador doméstico (empregado doméstico, cuidados, babá)', \
     'Militar', \
     'Policia! militar ou bombeiro militar', \
     'Empregado do setor privado', \
     'Empregado do setor público', \
     'Empregador', \
     'Autônomo (Conta própria)', \
     'Trabalhador familiar não remunerado em ajuda a parente', \
     'Estava fora do mercado de trabalho', 'N/A'])

df['TIPO_EMPREGO'].head()
```

Out[34]: 0      Empregado do setor privado  
1      Autônomo (Conta própria)  
2                      N/A  
3                      N/A  
4                      N/A  
Name: TIPO\_EMPREGO, dtype: category  
Categories (10, object): ['Trabalhador doméstico (empregado doméstico, c...', 'Militar', 'Policia! milita  
r ou bombeiro militar', 'Empregado do setor privado', ..., 'Autônomo (Conta própria)', 'Trabalhador fami  
liar não remunerado em ajuda a ...', 'Estava fora do mercado de trabalho', 'N/A']

• Convertendo a variável **CARTEIRA\_ASSINADA (C007B)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

In [35]: *# Vendo as primeiras linhas da coluna CARTEIRA\_ASSINADA*

```
df['CARTEIRA_ASSINADA'].head() #Outra forma: df.CARTEIRA_ASSINADA.head()
```

Out[35]: 0      1.0  
1      99.0  
2      99.0  
3      99.0  
4      99.0  
Name: CARTEIRA\_ASSINADA, dtype: float64

```
In [36]: # Valor Mínimo e Valor Máximo

# df.CARTEIRA_ASSINADA.describe()

print(df['CARTEIRA_ASSINADA'].min())
print(df['CARTEIRA_ASSINADA'].max())

1.0
99.0
```

```
In [37]: # Descobrindo os valores únicos da variável CARTEIRA_ASSINADA

ValoresOrdenados = df['CARTEIRA_ASSINADA'].unique()
ValoresOrdenados.sort()
ValoresOrdenados
```

```
Out[37]: array([ 1.,  2.,  3., 99.])
```

```
In [38]: # Convertendo a variável CARTEIRA_ASSINADA para categórica onde: 1=Sim, carteira de trab assinada; 2=Sim,
df['CARTEIRA_ASSINADA'] = df['CARTEIRA_ASSINADA'].astype('category')
df['CARTEIRA_ASSINADA'] = df['CARTEIRA_ASSINADA'].cat.rename_categories(['Sim, carteira de trab assinada',
df['CARTEIRA_ASSINADA'].head()
```

```
Out[38]: 0    Sim, carteira de trab assinada
1                N/A
2                N/A
3                N/A
4                N/A
Name: CARTEIRA_ASSINADA, dtype: category
Categories (4, object): ['Sim, carteira de trab assinada', 'Sim, servidor púb estatutário', 'Não', 'N/A']
```

• Convertendo a variável **SOMENTE\_BENEFICIO (C0103)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [39]: # Vendo as primeiras linhas da coluna SOMENTE_BENEFICIO

df['SOMENTE_BENEFICIO'].head() #Outra forma: df.SOMENTE_BENEFICIO.head()
```

```
Out[39]: 0    99.0
1    99.0
2    99.0
3    99.0
4    99.0
Name: SOMENTE_BENEFICIO, dtype: float64
```

```
In [40]: # Valor Mínimo e Valor Máximo

# df.SOMENTE_BENEFICIO.describe()

print(df['SOMENTE_BENEFICIO'].min())
print(df['SOMENTE_BENEFICIO'].max())

3.0
99.0
```

```
In [41]: # Descobrindo os valores únicos da variável SOMENTE_BENEFICIO

ValoresOrdenados = df['SOMENTE_BENEFICIO'].unique()
ValoresOrdenados.sort()
ValoresOrdenados
```

```
Out[41]: array([ 3., 99.])
```

```
In [42]: # Convertendo a variável SOMENTE_BENEFICIO para categórica onde 1=Sim; 2=N/A
```

```
df['SOMENTE_BENEFICIO'] = df['SOMENTE_BENEFICIO'].astype('category')
df['SOMENTE_BENEFICIO'] = df['SOMENTE_BENEFICIO'].cat.rename_categories(['Sim', 'N/A'])
df['SOMENTE_BENEFICIO'].head()
```

```
Out[42]: 0    N/A
         1    N/A
         2    N/A
         3    N/A
         4    N/A
         Name: SOMENTE_BENEFICIO, dtype: category
         Categories (2, object): ['Sim', 'N/A']
```

• Convertendo a variável **NAO\_REMUNERADO (C0104)** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [43]: # Vendo as primeiras Linhas da coluna NAO_REMUNERADO
```

```
df['NAO_REMUNERADO'].head() #Outra forma: df.NAO_REMUNERADO.head()
```

```
Out[43]: 0    99.0
         1    99.0
         2    99.0
         3    99.0
         4    99.0
         Name: NAO_REMUNERADO, dtype: float64
```

```
In [44]: # Valor Mínimo e Valor Máximo
```

```
# df.NAO_REMUNERADO.describe()

print(df['NAO_REMUNERADO'].min())
print(df['NAO_REMUNERADO'].max())
```

```
4.0
99.0
```

```
In [45]: # Descobrindo os valores únicos da variável NAO_REMUNERADO
```

```
ValoresOrdenados = df['NAO_REMUNERADO'].unique()
ValoresOrdenados.sort()
ValoresOrdenados
```

```
Out[45]: array([ 4., 99.])
```

```
In [46]: # Convertendo a variável NAO_REMUNERADO para categórica onde 1=Sim; 2=N/A
```

```
df['NAO_REMUNERADO'] = df['NAO_REMUNERADO'].astype('category')
df['NAO_REMUNERADO'] = df['NAO_REMUNERADO'].cat.rename_categories(['Sim', 'N/A'])
df['NAO_REMUNERADO'].head()
```

```
Out[46]: 0    N/A
         1    N/A
         2    N/A
         3    N/A
         4    N/A
         Name: NAO_REMUNERADO, dtype: category
         Categories (2, object): ['Sim', 'N/A']
```

• Convertendo a variável **UF** para o tipo "category" (qualitativa nominal) e fazendo o de-para:

```
In [47]: # Vendo as primeiras Linhas da coluna UF
```

```
df['UF'].head() #Outra forma: df.UF.head()
```

```
Out[47]: 0    11
         1    11
         2    11
         3    11
         4    11
         Name: UF, dtype: int64
```

```
In [48]: # Valor Mínimo e Valor Máximo
```

```
# df.UF.describe()

print(df['UF'].min())
print(df['UF'].max())
```

```
11
53
```

```
In [49]: # Convertendo a variável UF para categórica, conforme dicionário de UF's do IBGE
```

```
df['UF'] = df['UF'].astype('category')
df['UF'] = df['UF'].cat.rename_categories(['RO', 'AC', 'AM', 'RR', 'PA', 'AP', 'TO', 'MA', 'PI', 'CE', 'RN',
                                          'PE', 'AL', 'SE', 'BA', 'MG', 'ES', 'RJ', 'SP', 'PR', 'SC', 'PB',
                                          'MT', 'GO', 'DF'])

df['UF'].head()
```

```
Out[49]: 0    RO
1    RO
2    RO
3    RO
4    RO
Name: UF, dtype: category
Categories (27, object): ['RO', 'AC', 'AM', 'RR', ..., 'MS', 'MT', 'GO', 'DF']
```

#### 4.5) MUDANDO O TIPO DE ALGUMAS VARIÁVEIS:

```
In [50]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381270 entries, 0 to 381269
Data columns (total 26 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   UF                                         381270 non-null  category
1   V1022                                     381270 non-null  int64
2   V1023                                     381270 non-null  int64
3   IDADE                                     381270 non-null  category
4   SEXO                                       381270 non-null  category
5   COR                                       381270 non-null  category
6   ESCOLARIDADE                             381270 non-null  category
7   ESTAVA_TRABALHANDO                       381270 non-null  category
8   TIPO_EMPREGO                             381270 non-null  category
9   CARTEIRA_ASSINADA                       381270 non-null  category
10  C007C                                     381270 non-null  float64
11  C007D                                     381270 non-null  float64
12  C010                                     381270 non-null  float64
13  SALARIO                                   381270 non-null  float64
14  VALOR_TICKET-CESTA                       381270 non-null  float64
15  SOMENTE_BENEFICIO                       381270 non-null  category
16  NAO_REMUNERADO                         381270 non-null  category
17  C014                                     381270 non-null  float64
18  SOMA_APOSENTADORIAS+PENSAO             381270 non-null  float64
19  D0023                                     381270 non-null  float64
20  SOMA_BOLSAS_FAMILIA                     381270 non-null  float64
21  SOMA_BPC-LOAS                           381270 non-null  float64
22  AUXILIO_EMERGENCIAL                     381270 non-null  int64
23  SOMA_SEGUROS_DESEMPREGO                 381270 non-null  float64
24  D0073                                     381270 non-null  float64
25  COUNT                                   381270 non-null  int64
dtypes: category(10), float64(12), int64(4)
memory usage: 50.2 MB
```

• Deixando as variáveis de dinheiro como tipo "float64" (Boa Prática) e as demais como "Category" ou "int64", conforme o caso:

```
In [51]: df['V1022'] = df['V1022'].astype('category')
df['V1023'] = df['V1023'].astype('category')
df['C007C'] = df['C007C'].astype('category')
df['C007D'] = df['C007D'].astype('category')
df['C010'] = df['C010'].astype('category')
df['C014'] = df['C014'].astype('category')
```

• Convertendo a variável dependente "TAG" (OU RÓTULO) para o tipo "int32", apenas para deixarmos ela de fora do One-Hot Encoding posteriormente:

```
In [52]: df['AUXILIO_EMERGENCIAL'] = df['AUXILIO_EMERGENCIAL'].astype('int')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381270 entries, 0 to 381269
Data columns (total 26 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   UF                                           381270 non-null  category
1   V1022                                        381270 non-null  category
2   V1023                                        381270 non-null  category
3   IDADE                                       381270 non-null  category
4   SEXO                                        381270 non-null  category
5   COR                                         381270 non-null  category
6   ESCOLARIDADE                              381270 non-null  category
7   ESTAVA_TRABALHANDO                        381270 non-null  category
8   TIPO_EMPREGO                               381270 non-null  category
9   CARTEIRA_ASSINADA                         381270 non-null  category
10  C007C                                       381270 non-null  category
11  C007D                                       381270 non-null  category
12  C010                                        381270 non-null  category
13  SALARIO                                    381270 non-null  float64
14  VALOR_TICKET-CESTA                        381270 non-null  float64
15  SOMENTE_BENEFICIO                         381270 non-null  category
16  NAO_REMUNERADO                           381270 non-null  category
17  C014                                       381270 non-null  category
18  SOMA_APOSENTADORIAS+PENSAO               381270 non-null  float64
19  D0023                                      381270 non-null  float64
20  SOMA_BOLSAS_FAMILIA                      381270 non-null  float64
21  SOMA_BPC-LOAS                            381270 non-null  float64
22  AUXILIO_EMERGENCIAL                      381270 non-null  int32
23  SOMA_SEGUROS_DESEMPREGO                  381270 non-null  float64
24  D0073                                      381270 non-null  float64
25  COUNT                                     381270 non-null  int64
dtypes: category(16), float64(8), int32(1), int64(1)
memory usage: 33.5 MB
```

#### 4.6) ANONIMIZAÇÃO: (N/A)

```
In [53]: # OS DADOS QUE O IBGE DISPONIBILIZA JÁ SÃO ANONIMIZADOS!
```

#### 4.7) VERIFICAÇÃO DE DADOS MISSING E TRATANDO-OS/ REMOVENDO-OS:

```
In [54]: print(df.shape)
df.isna().sum()
```

```
(381270, 26)
```

```
Out[54]: UF                                0
V1022                                       0
V1023                                       0
IDADE                                       0
SEXO                                        0
COR                                         0
ESCOLARIDADE                             0
ESTAVA_TRABALHANDO                       0
TIPO_EMPREGO                             0
CARTEIRA_ASSINADA                         0
C007C                                      0
C007D                                      0
C010                                       0
SALARIO                                   0
VALOR_TICKET-CESTA                       0
SOMENTE_BENEFICIO                         0
NAO_REMUNERADO                           0
C014                                       0
SOMA_APOSENTADORIAS+PENSAO               0
D0023                                      0
SOMA_BOLSAS_FAMILIA                      0
SOMA_BPC-LOAS                            0
AUXILIO_EMERGENCIAL                      0
SOMA_SEGUROS_DESEMPREGO                  0
D0073                                      0
COUNT                                   0
dtype: int64
```

### 5) Análise Exploratória/ Resumo Estatístico:

#### 5.1) GRÁFICO: "AUXÍLIO EMERGENCIAL P/ UNIDADE DA FEDERAÇÃO"

• Query (Tabela):

```
In [55]: # Criando um dataframe para conferir pessoas que recebem Auxílio Emergencial

auxilio_por_UF = df[['UF', 'AUXILIO_EMERGENCIAL']][df['AUXILIO_EMERGENCIAL']==1].\
    groupby('UF').count().\
    sort_values(by='UF', ascending=True)

auxilio_por_UF
```

Out[55]:

| AUXILIO_EMERGENCIAL |       |
|---------------------|-------|
| UF                  |       |
| RO                  | 2732  |
| AC                  | 3942  |
| AM                  | 6590  |
| RR                  | 2930  |
| PA                  | 7694  |
| AP                  | 2253  |
| TO                  | 3098  |
| MA                  | 12869 |
| PI                  | 5514  |
| CE                  | 10917 |
| RN                  | 5203  |
| PB                  | 6111  |
| PE                  | 10093 |
| AL                  | 8401  |
| SE                  | 5540  |
| BA                  | 11896 |
| MG                  | 16223 |
| ES                  | 7083  |
| RJ                  | 11570 |
| SP                  | 13770 |
| PR                  | 7786  |
| SC                  | 6404  |
| RS                  | 6831  |
| MS                  | 3808  |
| MT                  | 5350  |
| GO                  | 6699  |
| DF                  | 2377  |

In [56]: *# Criando um dataframe para conferir o total de entrevistados*

```
entrevistado_por_UF = df[['UF', 'COUNT']][df['COUNT']==1].\
    groupby('UF').count().\
    sort_values(by='UF', ascending=True)

entrevistado_por_UF
```

Out[56]:

|    | COUNT |
|----|-------|
| UF |       |
| RO | 5353  |
| AC | 6656  |
| AM | 9835  |
| RR | 4992  |
| PA | 11350 |
| AP | 3245  |
| TO | 5335  |
| MA | 17883 |
| PI | 8350  |
| CE | 16828 |
| RN | 8235  |
| PB | 9639  |
| PE | 15762 |
| AL | 12101 |
| SE | 8519  |
| BA | 17946 |
| MG | 34541 |
| ES | 14828 |
| RJ | 28536 |
| SP | 35430 |
| PR | 19861 |
| SC | 23448 |
| RS | 22061 |
| MS | 8775  |
| MT | 11403 |
| GO | 13703 |
| DF | 6655  |

```
In [57]: # Unindo os dois dataframes anteriores
```

```
df_unido = pd.merge(auxilio_por_UF, entrevistado_por_UF, on='UF', how='outer')
df_unido
```

```
Out[57]:
```

|    | AUXILIO_EMERGENCIAL | COUNT |
|----|---------------------|-------|
| UF |                     |       |
| RO | 2732                | 5353  |
| AC | 3942                | 6656  |
| AM | 6590                | 9835  |
| RR | 2930                | 4992  |
| PA | 7694                | 11350 |
| AP | 2253                | 3245  |
| TO | 3098                | 5335  |
| MA | 12869               | 17883 |
| PI | 5514                | 8350  |
| CE | 10917               | 16828 |
| RN | 5203                | 8235  |
| PB | 6111                | 9639  |
| PE | 10093               | 15762 |
| AL | 8401                | 12101 |
| SE | 5540                | 8519  |
| BA | 11896               | 17946 |
| MG | 16223               | 34541 |
| ES | 7083                | 14828 |
| RJ | 11570               | 28536 |
| SP | 13770               | 35430 |
| PR | 7786                | 19861 |
| SC | 6404                | 23448 |
| RS | 6831                | 22061 |
| MS | 3808                | 8775  |
| MT | 5350                | 11403 |
| GO | 6699                | 13703 |
| DF | 2377                | 6655  |

```
In [58]: # Calculando o percentual de pessoas c/ auxilio emergencial dentre os entrevistados
df_unido['%_Pessoas_Auxilio'] = (df_unido['AUXILIO_EMERGENCIAL'] / df_unido['COUNT']) * 100
df_unido = df_unido.sort_values(by='%_Pessoas_Auxilio', ascending=False) # Para o gráfico ficar ordenado c

# Visualizando o resultado
df_unido = df_unido.reset_index()
df_unido['UF'] = df_unido['UF'].astype(str) # Convertei a col 'UF' p/ string, de modo q a ordenação das bo
df_unido
```

```
Out[58]:
```

|    | UF | AUXILIO_EMERGENCIAL | COUNT | %_Pessoas_Auxilio |
|----|----|---------------------|-------|-------------------|
| 0  | MA | 12869               | 17883 | 71.962199         |
| 1  | AP | 2253                | 3245  | 69.429892         |
| 2  | AL | 8401                | 12101 | 69.424015         |
| 3  | PA | 7694                | 11350 | 67.788546         |
| 4  | AM | 6590                | 9835  | 67.005592         |
| 5  | BA | 11896               | 17946 | 66.287752         |
| 6  | PI | 5514                | 8350  | 66.035928         |
| 7  | SE | 5540                | 8519  | 65.031107         |
| 8  | CE | 10917               | 16828 | 64.874019         |
| 9  | PE | 10093               | 15762 | 64.033752         |
| 10 | PB | 6111                | 9639  | 63.398693         |
| 11 | RN | 5203                | 8235  | 63.181542         |
| 12 | AC | 3942                | 6656  | 59.224760         |
| 13 | RR | 2930                | 4992  | 58.693910         |
| 14 | TO | 3098                | 5335  | 58.069353         |
| 15 | RO | 2732                | 5353  | 51.036802         |
| 16 | GO | 6699                | 13703 | 48.887105         |
| 17 | ES | 7083                | 14828 | 47.767737         |
| 18 | MG | 16223               | 34541 | 46.967372         |
| 19 | MT | 5350                | 11403 | 46.917478         |
| 20 | MS | 3808                | 8775  | 43.396011         |
| 21 | RJ | 11570               | 28536 | 40.545276         |
| 22 | PR | 7786                | 19861 | 39.202457         |
| 23 | SP | 13770               | 35430 | 38.865368         |
| 24 | DF | 2377                | 6655  | 35.717506         |
| 25 | RS | 6831                | 22061 | 30.964145         |
| 26 | SC | 6404                | 23448 | 27.311498         |

• Plot:

In [59]:

```
# Área de plotagem
plt.figure(figsize=(12, 5))
sns.set_theme(style="whitegrid")

# Gráfico de barras
ax = sns.barplot(
    data=df_unido,
    x='UF',
    y='%_Pessoas_Auxilio'
)

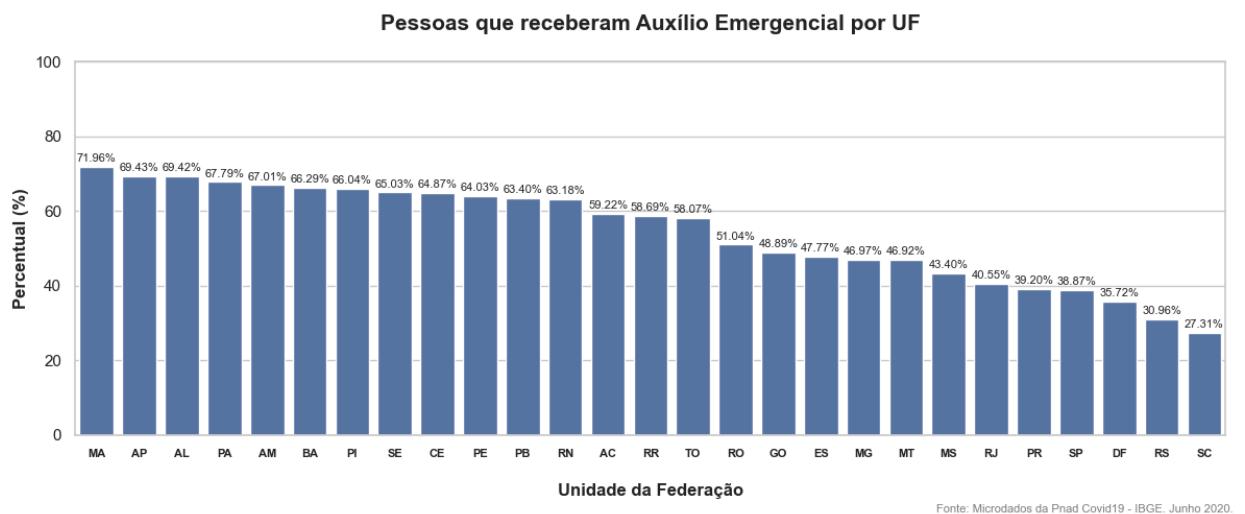
# Adiciona os rótulos de percentual em cada barra
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f%', label_type='edge', fontsize=8, padding=2)

# Ajustes de Layout e estilo
ax.set_xlabel("\nUnidade da Federação", fontsize=12, fontweight='bold')
ax.set_ylabel("Percentual (%)", fontsize=12, fontweight='bold')
ax.set_title("Pessoas que receberam Auxílio Emergencial por UF\n", \
            fontsize=15, fontweight='bold')
ax.set_xticklabels(df_unido['UF'], fontsize=8, fontweight='bold')
plt.ylim(0, 100)
plt.tight_layout()

# Adiciona a fonte como texto no rodapé
plt.figtext(0.99, 0.01, "Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.",
            horizontalalignment='right', fontsize=8, color='gray')

# Salva o gráfico em PNG
#plt.savefig("auxilio_domicilio_ssa.png", dpi=120)
#plt.show()
```

Out[59]: Text(0.99, 0.01, 'Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.')



## 5.2) GRÁFICO: "AUXÍLIO EMERGENCIAL P/ SEXO E COR":

• Query (Tabela):

```
In [60]: # Criando um dataframe para conferir pessoas que recebem Auxílio Emergencial

auxilio_cor_sexo = df[['COR', 'SEXO', 'AUXILIO_EMERGENCIAL']][df['AUXILIO_EMERGENCIAL']==1].\
    groupby(['COR', 'SEXO']).count().\
    sort_values(by='COR', ascending=True)

auxilio_cor_sexo
```

```
Out[60]:
```

| AUXILIO_EMERGENCIAL |        |       |
|---------------------|--------|-------|
| COR                 | SEXO   |       |
| Branca              | Homem  | 30357 |
|                     | Mulher | 33155 |
| Preta               | Homem  | 8718  |
|                     | Mulher | 8756  |
| Amarela             | Homem  | 445   |
|                     | Mulher | 512   |
| Parda               | Homem  | 54012 |
|                     | Mulher | 56765 |
| Indígena            | Homem  | 417   |
|                     | Mulher | 507   |
| Ignorado            | Homem  | 24    |
|                     | Mulher | 16    |

```
In [61]: # Criando um dataframe para conferir o total de entrevistados

entrevistado_cor_sexo = df[['COR', 'SEXO', 'COUNT']][df['COUNT']==1].\
    groupby(['COR', 'SEXO']).count().\
    sort_values(by='COR', ascending=True)

entrevistado_cor_sexo
```

```
Out[61]:
```

| COUNT    |        |       |
|----------|--------|-------|
| COR      | SEXO   |       |
| Branca   | Homem  | 75321 |
|          | Mulher | 84566 |
| Preta    | Homem  | 15566 |
|          | Mulher | 15743 |
| Amarela  | Homem  | 1107  |
|          | Mulher | 1242  |
| Parda    | Homem  | 90358 |
|          | Mulher | 95782 |
| Indígena | Homem  | 682   |
|          | Mulher | 805   |
| Ignorado | Homem  | 57    |
|          | Mulher | 41    |

In [62]: *# Unindo os dois dataframes anteriores*

```
df_unido = pd.merge(auxilio_cor_sexo, entrevistado_cor_sexo, on=['COR', 'SEXO'], how='outer')
df_unido
```

Out[62]:

|          |        | AUXILIO_EMERGENCIAL | COUNT |
|----------|--------|---------------------|-------|
| COR      | SEXO   |                     |       |
| Branca   | Homem  | 30357               | 75321 |
|          | Mulher | 33155               | 84566 |
| Preta    | Homem  | 8718                | 15566 |
|          | Mulher | 8756                | 15743 |
| Amarela  | Homem  | 445                 | 1107  |
|          | Mulher | 512                 | 1242  |
| Parda    | Homem  | 54012               | 90358 |
|          | Mulher | 56765               | 95782 |
| Indígena | Homem  | 417                 | 682   |
|          | Mulher | 507                 | 805   |
| Ignorado | Homem  | 24                  | 57    |
|          | Mulher | 16                  | 41    |

In [63]: *# Calculando o percentual de pessoas com auxílio emergencial dentre os entrevistados*

```
df_unido['%_Pessoas_Auxilio'] = (df_unido['AUXILIO_EMERGENCIAL'] / df_unido['COUNT']) * 100
```

*# Visualizando o resultado*

```
df_unido = df_unido.reset_index()
df_unido
```

Out[63]:

|    | COR      | SEXO   | AUXILIO_EMERGENCIAL | COUNT | %_Pessoas_Auxilio |
|----|----------|--------|---------------------|-------|-------------------|
| 0  | Branca   | Homem  | 30357               | 75321 | 40.303501         |
| 1  | Branca   | Mulher | 33155               | 84566 | 39.206064         |
| 2  | Preta    | Homem  | 8718                | 15566 | 56.006681         |
| 3  | Preta    | Mulher | 8756                | 15743 | 55.618370         |
| 4  | Amarela  | Homem  | 445                 | 1107  | 40.198735         |
| 5  | Amarela  | Mulher | 512                 | 1242  | 41.223833         |
| 6  | Parda    | Homem  | 54012               | 90358 | 59.775559         |
| 7  | Parda    | Mulher | 56765               | 95782 | 59.264789         |
| 8  | Indígena | Homem  | 417                 | 682   | 61.143695         |
| 9  | Indígena | Mulher | 507                 | 805   | 62.981366         |
| 10 | Ignorado | Homem  | 24                  | 57    | 42.105263         |
| 11 | Ignorado | Mulher | 16                  | 41    | 39.024390         |

In [64]: *# Exluindo os índices correspondentes aos 'Ignorados'*

```
df_unido = df_unido.drop(df_unido[(df_unido['COR'] == 'Ignorado')].index)
df_unido.reset_index(drop=True, inplace=True)
df_unido['COR'] = df_unido['COR'].cat.remove_unused_categories() # Para essa core ã apareca no gráfico!
df_unido
```

Out[64]:

|   | COR      | SEXO   | AUXILIO_EMERGENCIAL | COUNT | %_Pessoas_Auxilio |
|---|----------|--------|---------------------|-------|-------------------|
| 0 | Branca   | Homem  | 30357               | 75321 | 40.303501         |
| 1 | Branca   | Mulher | 33155               | 84566 | 39.206064         |
| 2 | Preta    | Homem  | 8718                | 15566 | 56.006681         |
| 3 | Preta    | Mulher | 8756                | 15743 | 55.618370         |
| 4 | Amarela  | Homem  | 445                 | 1107  | 40.198735         |
| 5 | Amarela  | Mulher | 512                 | 1242  | 41.223833         |
| 6 | Parda    | Homem  | 54012               | 90358 | 59.775559         |
| 7 | Parda    | Mulher | 56765               | 95782 | 59.264789         |
| 8 | Indígena | Homem  | 417                 | 682   | 61.143695         |
| 9 | Indígena | Mulher | 507                 | 805   | 62.981366         |

• Plot:

In [65]: # Gráfico

```
# Área de Plotagem
plt.figure(figsize=(10, 5))
sns.set_theme(style="whitegrid")

# Gráfico de barras agrupadas
ax = sns.barplot(
    data=df_unido,
    x='SEXO',
    y='%_Pessoas_Auxilio',
    hue='COR',
    palette=["#c1b9ff", "#636e72", "#fdbc6e", "#0984e3", "#ff7675"]
)

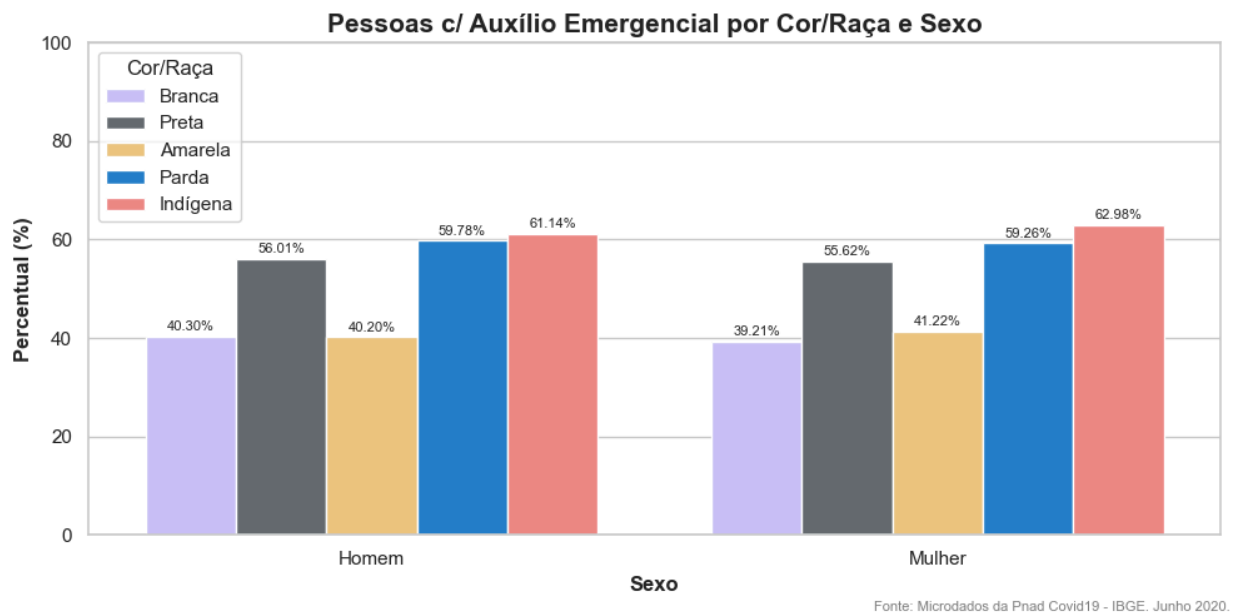
# Adiciona os rótulos de percentual em cada barra
for container in ax.containers:
    ax.bar_label(container, fmt='%0.2f%', label_type='edge', fontsize=8, padding=2)

# Ajustes de Layout e estilo
ax.set_xlabel("Sexo", fontsize=12, fontweight='bold')
ax.set_ylabel("Percentual (%)", fontsize=12, fontweight='bold')
ax.set_title("Pessoas c/ Auxílio Emergencial por Cor/Raça e Sexo", \
             fontsize=15, fontweight='bold')
ax.legend(title="Cor/Raça", loc='upper left')
plt.ylim(0, 100)
plt.tight_layout()

# Adiciona a fonte como texto no rodapé
plt.figtext(0.99, 0.01, "Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.",
            horizontalalignment='right', fontsize=8, color='gray')

# Salva o gráfico em PNG
#plt.savefig("auxilio_cor_sexo_ssa.png", dpi=120)
#plt.show()
```

Out[65]: Text(0.99, 0.01, 'Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.')



Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.

### 5.3) GRÁFICO: "AUXÍLIO EMERGENCIAL P/ FAIXA SALARIAL"

• Criando uma nova variável no final do dataframe 'df' p/ segmentar as faixas salariais:

In [66]: # Adicionando uma nova coluna ao dataframe

```
df['FAIXA_SALARIO'] = df['SALARIO']
```

In [67]: *# Vendo as primeiras linhas da coluna FAIXA\_SALARIO*

```
df['FAIXA_SALARIO'].head()
```

Out[67]:

|   |        |
|---|--------|
| 0 | 1045.0 |
| 1 | 1045.0 |
| 2 | 0.0    |
| 3 | 0.0    |
| 4 | 0.0    |

Name: FAIXA\_SALARIO, dtype: float64

In [68]: *# Valor Mínimo e Valor Máximo*

```
# df.FAIXA_SALARIO.describe()

print(df['FAIXA_SALARIO'].min())
print(df['FAIXA_SALARIO'].max())
```

0.0  
200000.0

In [69]: *# Convertendo a variável FAIXA\_SALARIO para categórica, onde:*

```
#
# de 0 a 1044 = "Menos de um salário mínimo",
# de 1045 a 2090 = "Entre 1 e 2",
# de 2091 a 3135 = "Entre 2 e 3",
# de 3136 a 4180 = "Entre 3 e 4",
# de 4181 a 5225 = "Entre 4 e 5"
# + que 5226 = "Mais de 5"

df['FAIXA_SALARIO'] = pd.cut(df['FAIXA_SALARIO'], \
    bins=[-1, 1044, 2090, 3135, 4180, 5225, 250000], \
    labels=['Menos de um salário mínimo', \
            'Entre 1 e 2', \
            'Entre 2 e 3', \
            'Entre 3 e 4', \
            'Entre 4 e 5', \
            'Mais de 5'])

df['FAIXA_SALARIO'].head()
```

Out[69]:

|   |                            |
|---|----------------------------|
| 0 | Entre 1 e 2                |
| 1 | Entre 1 e 2                |
| 2 | Menos de um salário mínimo |
| 3 | Menos de um salário mínimo |
| 4 | Menos de um salário mínimo |

Name: FAIXA\_SALARIO, dtype: category  
Categories (6, object): ['Menos de um salário mínimo' < 'Entre 1 e 2' < 'Entre 2 e 3' < 'Entre 3 e 4' < 'Entre 4 e 5' < 'Mais de 5']

#### • Query (Tabela):

In [70]: *# Criando um dataframe para conferir pessoas que recebem Auxílio Emergencial*

```
auxilio_renda = df[['FAIXA_SALARIO', 'AUXILIO_EMERGENCIAL']][df['AUXILIO_EMERGENCIAL']==1].\
    groupby('FAIXA_SALARIO').count().\
    sort_values(by='FAIXA_SALARIO', ascending=True)

auxilio_renda
```

Out[70]:

| FAIXA_SALARIO              | AUXILIO_EMERGENCIAL |
|----------------------------|---------------------|
| Menos de um salário mínimo | 153042              |
| Entre 1 e 2                | 32305               |
| Entre 2 e 3                | 5257                |
| Entre 3 e 4                | 1508                |
| Entre 4 e 5                | 745                 |
| Mais de 5                  | 827                 |

```
In [71]: # Criando um dataframe para conferir o total de entrevistados

entrevistado_renda = df[['FAIXA_SALARIO', 'COUNT']][df['COUNT']==1].\
    groupby('FAIXA_SALARIO').count().\
    sort_values(by='FAIXA_SALARIO', ascending=True)

entrevistado_renda
```

```
Out[71]:
```

|                            | COUNT  |
|----------------------------|--------|
| FAIXA_SALARIO              |        |
| Menos de um salário mínimo | 271030 |
| Entre 1 e 2                | 71666  |
| Entre 2 e 3                | 17403  |
| Entre 3 e 4                | 7087   |
| Entre 4 e 5                | 4918   |
| Mais de 5                  | 9166   |

```
In [72]: # Unindo os dois dataframes anteriores

df_unido = pd.merge(auxilio_renda, entrevistado_renda, on='FAIXA_SALARIO', how='outer')
df_unido
```

```
Out[72]:
```

|                            | AUXILIO_EMERGENCIAL | COUNT  |
|----------------------------|---------------------|--------|
| FAIXA_SALARIO              |                     |        |
| Menos de um salário mínimo | 153042              | 271030 |
| Entre 1 e 2                | 32305               | 71666  |
| Entre 2 e 3                | 5257                | 17403  |
| Entre 3 e 4                | 1508                | 7087   |
| Entre 4 e 5                | 745                 | 4918   |
| Mais de 5                  | 827                 | 9166   |

```
In [73]: # Calculando o percentual de pessoas c/ auxilio emergencial dentre os entrevistados
df_unido['%Pessoas_Auxilio'] = (df_unido['AUXILIO_EMERGENCIAL'] / df_unido['COUNT']) * 100

# Visualizando o resultado
df_unido = df_unido.reset_index()
df_unido
```

```
Out[73]:
```

|   | FAIXA_SALARIO              | AUXILIO_EMERGENCIAL | COUNT  | %Pessoas_Auxilio |
|---|----------------------------|---------------------|--------|------------------|
| 0 | Menos de um salário mínimo | 153042              | 271030 | 56.466812        |
| 1 | Entre 1 e 2                | 32305               | 71666  | 45.077164        |
| 2 | Entre 2 e 3                | 5257                | 17403  | 30.207435        |
| 3 | Entre 3 e 4                | 1508                | 7087   | 21.278397        |
| 4 | Entre 4 e 5                | 745                 | 4918   | 15.148434        |
| 5 | Mais de 5                  | 827                 | 9166   | 9.022474         |

• Plot:

In [74]:

```
# Área de plotagem
plt.figure(figsize=(10, 5))
sns.set_theme(style="whitegrid")

# Gráfico de barras horizontal (barh)
ax = sns.barplot(
    data=df_unido,
    y='FAIXA_SALARIO',
    x='%_Pessoas_Auxilio',
    palette=["#00b894", "#ff7675", "#0984e3", "#c1b9ff", "#fdbc6e", "#636e72"],
    orient='h'
)

# Remove as Linhas de grade
ax.grid(False)

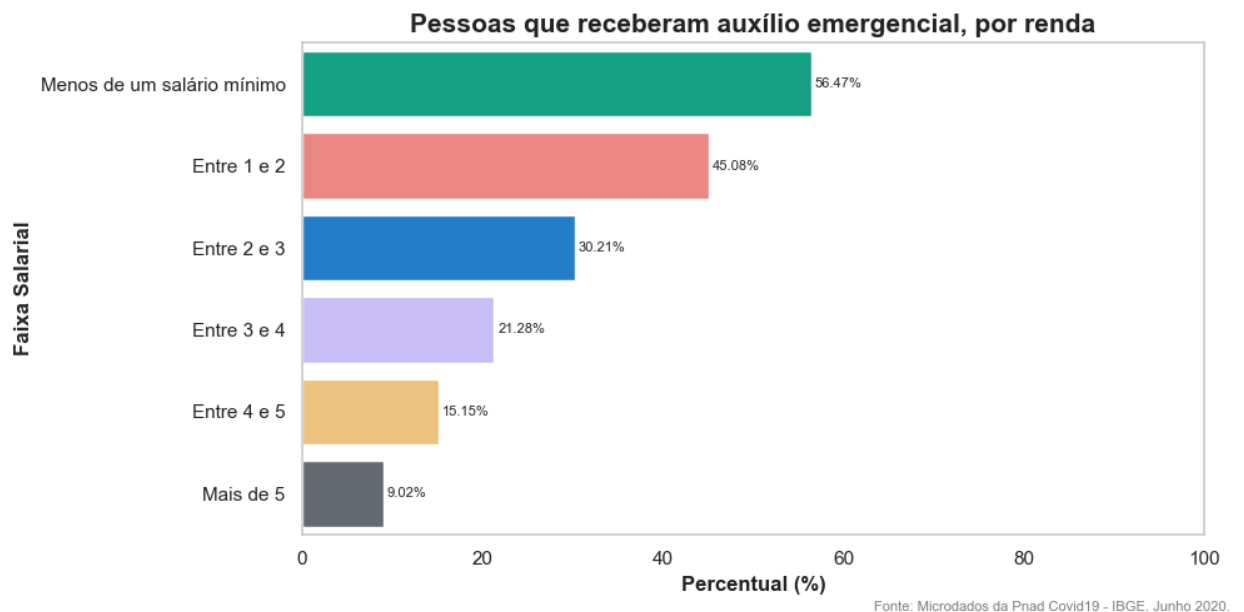
# Adiciona os rótulos de percentual em cada barra
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f%', fontsize=8, padding=2)

# Ajustes de Layout e estilo
ax.set_xlabel("Percentual (%)", fontsize=12, fontweight='bold')
ax.set_ylabel("Faixa Salarial", fontsize=12, fontweight='bold')
ax.set_title("Pessoas que receberam auxílio emergencial, por renda", fontsize=15, \
             fontweight='bold')
ax.set_xlim(0, 100)
plt.tight_layout()

# Adiciona a fonte como texto no rodapé
plt.figtext(0.99, 0.01, "Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.",
            horizontalalignment='right', fontsize=8, color='gray')

# Salva o gráfico em PNG
#plt.savefig("auxilio_renda_ssa.png", dpi=120)
#plt.show()
```

Out[74]: Text(0.99, 0.01, 'Fonte: Microdados da Pnad Covid19 - IBGE. Junho 2020.')



• Exluindo a variável c/ as faixas salariais e tb a variável contadora 'COUNT' p/ não interferir no modelo preditivo:

In [75]: df = df.drop(['COUNT', 'FAIXA\_SALARIO'], axis = 1)

## 5.4) VERIFICANDO SE A CLASSE PREDITORA ESTÁ DESBALANCEADA:

• Verificando a proporção dos que RECEBEM o Auxílio Emergencial e dos que NÃO RECEBEM:

```
In [76]: # Mostrando a quantidade de pessoas COM AUXÍLIO EMERGENCIAL (=1) e a quantidade de pessoas SEM AUXÍLIO EMERGENCIAL  
df['AUXILIO_EMERGENCIAL'].value_counts()
```

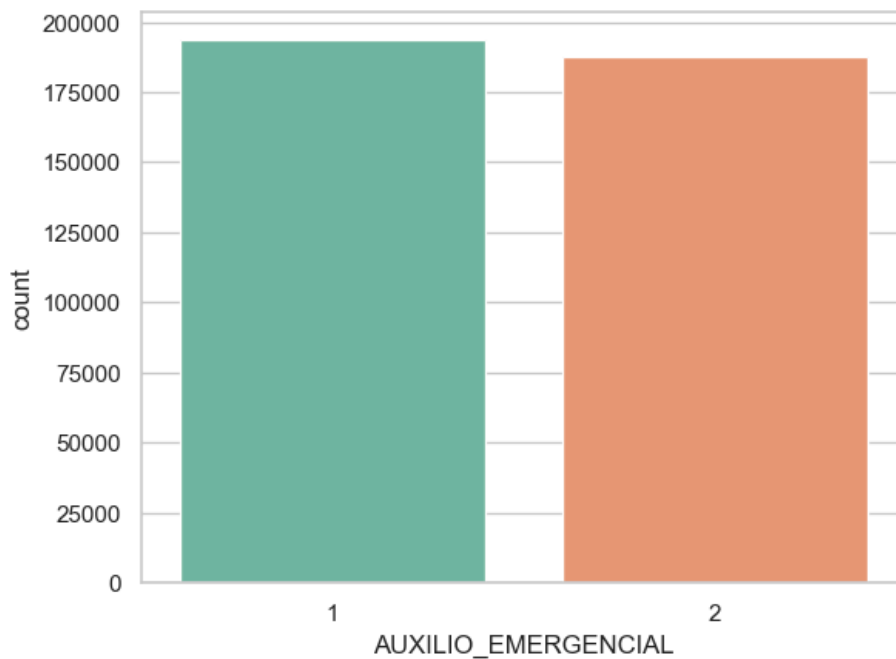
```
Out[76]: AUXILIO_EMERGENCIAL  
1      193684  
2      187586  
Name: count, dtype: int64
```

```
In [77]: # Mostrando a porcentagem dos que TÊM Auxílio Emergencial (classe preditora) e dos que NÃO TÊM  
df['AUXILIO_EMERGENCIAL'].value_counts(normalize=True) # O padrão é 'False' (valores absolutos)
```

```
Out[77]: AUXILIO_EMERGENCIAL  
1      0.507997  
2      0.492003  
Name: proportion, dtype: float64
```

```
In [78]: # Fazendo um gráfico para mostrar a quantidade de registros em cada classe  
sns.countplot(x='AUXILIO_EMERGENCIAL', data=df, palette='Set2')
```

```
Out[78]: <Axes: xlabel='AUXILIO_EMERGENCIAL', ylabel='count'>
```



## 5.5) OUTROS GRÁFICOS P/ ANÁLISE DAS DISTRIBUIÇÕES E OUTLIERS:

- Automatizando a criação de Boxplots (variáveis numéricas) e Gráf de Barras (categóricas):

In [79]: # *Análise Exploratória de Dados (EDA)*

# *Função criada p/ automatizar a criação dos gráficos*

def eda(dados):

for column in dados.columns:

# *Se a coluna for numérica (faremos um boxplot)*

if dados[column].dtype in ['int64', 'float64']:

# *Histograma e Boxplot*

fig, axes = plt.subplots(1, 2)

sns.histplot(dados[column], kde = True, ax = axes[0])

sns.boxplot(x = 'AUXILIO\_EMERGENCIAL', y = column, data = dados, ax = axes[1])

axes[0].set\_title(f'Distribuição de {column}')

axes[1].set\_title(f'{column} vs AUXILIO\_EMERGENCIAL')

plt.tight\_layout()

plt.show()

# *Se a coluna for categórica (faremos gráfico de barras)*

else:

# *Contagem de frequência e relação com AUXILIO\_EMERGENCIAL*

fig, axes = plt.subplots(1, 2)

sns.countplot(x = column, data = dados, ax = axes[0])

sns.countplot(x = column, hue = 'AUXILIO\_EMERGENCIAL', data = dados, ax = axes[1])

axes[0].set\_title(f'Distribuição de {column}')

axes[1].set\_title(f'{column} vs AUXILIO\_EMERGENCIAL')

plt.tight\_layout()

plt.show()

In [80]: # *Realizando a EDA (Análise Exploratória de Dados)*

# *Ao chamar a função criada na célula acima, estaremos criando um gráfico para cada coluna do dataframe.*

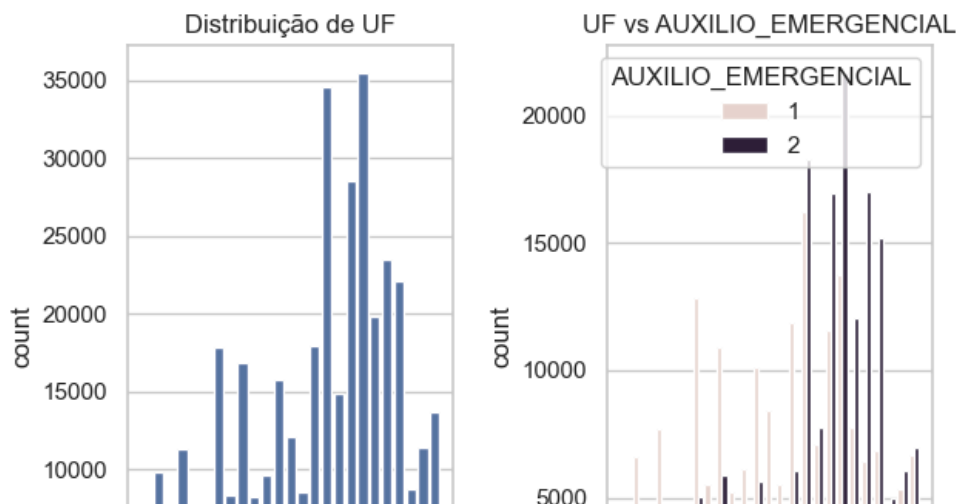
# *Dentre outras, isso nos ajudará a responder às seguintes perguntas:*

# *1) Quais colunas (variáveis) iremos lançar mão para seguir adiante com nosso modelo preditivo?*

# *2) Existe algum problema/anomalia ou outliers?*

# *3) Preciso investigar mais a fundo alguma variável?*

eda(df)



## 6) Divisão Treino-Teste:

In [81]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 381270 entries, 0 to 381269
Data columns (total 25 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   UF                                      381270 non-null  category
 1   V1022                                  381270 non-null  category
 2   V1023                                  381270 non-null  category
 3   IDADE                                  381270 non-null  category
 4   SEXO                                   381270 non-null  category
 5   COR                                    381270 non-null  category
 6   ESCOLARIDADE                          381270 non-null  category
 7   ESTAVA_TRABALHANDO                    381270 non-null  category
 8   TIPO_EMPREGO                           381270 non-null  category
 9   CARTEIRA_ASSINADA                      381270 non-null  category
10   C007C                                  381270 non-null  category
11   C007D                                  381270 non-null  category
12   C010                                    381270 non-null  category
13   SALARIO                                381270 non-null  float64
14   VALOR_TICKET-CESTA                     381270 non-null  float64
15   SOMENTE_BENEFICIO                      381270 non-null  category
16   NAO_REMUNERADO                         381270 non-null  category
17   C014                                    381270 non-null  category
18   SOMA_APOSENTADORIAS+PENSAO             381270 non-null  float64
19   D0023                                  381270 non-null  float64
20   SOMA_BOLSAS_FAMILIA                    381270 non-null  float64
21   SOMA_BPC-LOAS                          381270 non-null  float64
22   AUXILIO_EMERGENCIAL                    381270 non-null  int32
23   SOMA_SEGUROS_DESEMPREGO                381270 non-null  float64
24   D0073                                  381270 non-null  float64
dtypes: category(16), float64(8), int32(1)
memory usage: 30.5 MB
```

In [82]: `# Divisão em Treino e Teste`

```
valor_test_size = 0.3
X = df.drop(columns = ['AUXILIO_EMERGENCIAL'], axis = 1) # Matriz contendo "todo" o dataframe, menos a variável alvo
y = df['AUXILIO_EMERGENCIAL'] # Vetor contendo a variável target (rótulo)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = valor_test_size, random_state = 42)
```

In [83]: `# 'Recebem Auxilio' X 'Não recebem Auxilio' na tabela de Treino`

```
y_train.value_counts()
```

Out[83]: AUXILIO\_EMERGENCIAL  
1 135679  
2 131210  
Name: count, dtype: int64

In [84]: `# Mostrando em porcentagem`

```
y_train.value_counts(normalize=True) # O padrão é 'False' (valores absolutos)
```

Out[84]: AUXILIO\_EMERGENCIAL  
1 0.508372  
2 0.491628  
Name: proportion, dtype: float64

In [85]: `# Verificando o tamanho dos conjuntos de Treino e Teste`

```
X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

Out[85]: ((266889, 24), (114381, 24), (266889,), (114381,))

## 7) Pré-processamento/Padronização (Preparando p/ o algoritmo):

• Convertendo as variáveis categóricas em numéricas e usando o One-Hot Encoding:

```
In [86]: # Amostra dos dados
df.head()
```

Out[86]:

|   | UF | V1022 | V1023 | IDADE | SEXO   | COR    | ESCOLARIDADE                            | ESTAVA_TRABALHANDO | TIPO_EMPREGO               | CARTEIRA_ASSINADA              |
|---|----|-------|-------|-------|--------|--------|-----------------------------------------|--------------------|----------------------------|--------------------------------|
| 0 | RO | 1     | 1     | 18-49 | Homem  | Parda  | Médio Completo ou Superior Incompleto   | Sim                | Empregado do setor privado | Sim, carteira de trab assinada |
| 1 | RO | 1     | 1     | 18-49 | Mulher | Parda  | Superior Completo                       | Não                | Autônomo (Conta própria)   | N/A                            |
| 2 | RO | 1     | 1     | 12-17 | Homem  | Parda  | Sem instrução ou Fundamental Incompleto | N/A                | N/A                        | N/A                            |
| 3 | RO | 1     | 1     | 5-11  | Homem  | Parda  | Sem instrução ou Fundamental Incompleto | N/A                | N/A                        | N/A                            |
| 4 | RO | 1     | 1     | 50-64 | Mulher | Branca | Sem instrução ou Fundamental Incompleto | Não                | N/A                        | N/A                            |

5 rows × 25 columns

```
In [87]: # Selecionando as variáveis categóricas (e armazenando numa lista do pandas)
categorical_cols = df.select_dtypes(include = ['category']).columns
categorical_cols
```

```
Out[87]: Index(['UF', 'V1022', 'V1023', 'IDADE', 'SEXO', 'COR', 'ESCOLARIDADE',
               'ESTAVA_TRABALHANDO', 'TIPO_EMPREGO', 'CARTEIRA_ASSINADA', 'C007C',
               'C007D', 'C010', 'SOMENTE_BENEFICIO', 'NAO_REMUNERADO', 'C014'],
              dtype='object')
```

```
In [88]: # Aplicando One-Hot Encoding separadamente aos conjuntos de treino e teste

encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

# Obs: Está sendo criado o objeto 'encoder', como uma instância da classe OneHotEncoder() do sklearn
# Será usado para transformar variáveis categóricas em numéricas com o One-Hot Encoding

# sparse_output=False: o resultado será um DataFrame/array denso, não esparsa
# handle_unknown='ignore': categorias desconhecidas nos dados de teste serão ignoradas (não causam erro).
```

```
In [89]: # Treinando o objeto "encoder" com o conjunto de treino e transformando ambos (treino e teste)

X_train_encoded = pd.DataFrame(encoder.fit_transform(X_train[categorical_cols]))
X_test_encoded = pd.DataFrame(encoder.transform(X_test[categorical_cols]))
```

```
In [90]: X_train_encoded.head()
```

Out[90]:

|   | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | ... | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 |
| 1 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 1.0 | 0.0 | ... | 1.0 | 0.0 | 1.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 |
| 2 | 1.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 |
| 3 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 1.0 | ... | 1.0 | 0.0 | 1.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 |
| 4 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 |

5 rows × 141 columns

```
In [91]: X_test_encoded.head()
```

```
Out[91]:
```

|   | 0   | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | ... | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 1.0 | 0.0 | 1.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 |
| 1 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 1.0 | 0.0 | 1.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 |
| 2 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 1.0 | 0.0 | 1.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 1.0 | 0.0 |
| 3 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 1.0 | 0.0 | 1.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 |
| 4 | 1.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | ... | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 1.0 | 0.0 | 0.0 | 0.0 | 1.0 |

5 rows × 141 columns

```
In [92]: # Renomeando as colunas para corresponderem aos nomes das categorias
```

```
X_train_encoded.columns = encoder.get_feature_names_out(categorical_cols)
X_test_encoded.columns = encoder.get_feature_names_out(categorical_cols)
```

```
In [93]: X_train_encoded.head()
```

```
Out[93]:
```

|   | UF_AC | UF_AL | UF_AM | UF_AP | UF_BA | UF_CE | UF_DF | UF_ES | UF_GO | UF_MA | ... | C007D_99.0 | C010_1.0 | C010_99.0 | SOME |
|---|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|------------|----------|-----------|------|
| 0 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | ... | 0.0        | 1.0      | 0.0       |      |
| 1 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 1.0   | 0.0   | ... | 1.0        | 0.0      | 1.0       |      |
| 2 | 1.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | ... | 0.0        | 1.0      | 0.0       |      |
| 3 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 1.0   | ... | 1.0        | 0.0      | 1.0       |      |
| 4 | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | 0.0   | ... | 0.0        | 1.0      | 0.0       |      |

5 rows × 141 columns

```
In [94]: X_train_encoded.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 266889 entries, 0 to 266888
Columns: 141 entries, UF_AC to C014_99.0
dtypes: float64(141)
memory usage: 287.1 MB
```

```
In [95]: # Removendo as colunas categóricas originais e adicionando as codificadas nos dados de Treino
```

```
X_train_preprocessed = X_train.drop(categorical_cols, axis = 1).reset_index(drop = True)
X_train_preprocessed = pd.concat([X_train_preprocessed, X_train_encoded], axis = 1)
```

```
In [96]: # Visualizando as primeiras linhas dos dados de treino pré-processados
```

```
X_train_preprocessed.head()
```

```
Out[96]:
```

|   | SALARIO | VALOR_TICKET-CESTA | SOMA_APOSENTADORIAS+PENSAO | D0023 | SOMA_BOLSAS_FAMILIA | SOMA_BPC-LOAS | SOMA_SEGUROS |
|---|---------|--------------------|----------------------------|-------|---------------------|---------------|--------------|
| 0 | 1045.0  | 0.0                | 1045.0                     | 0.0   | 0.0                 | 0.0           |              |
| 1 | 0.0     | 0.0                | 1045.0                     | 0.0   | 0.0                 | 0.0           |              |
| 2 | 1100.0  | 0.0                | 0.0                        | 0.0   | 0.0                 | 1045.0        |              |
| 3 | 0.0     | 0.0                | 0.0                        | 0.0   | 212.0               | 0.0           |              |
| 4 | 800.0   | 0.0                | 2090.0                     | 0.0   | 0.0                 | 0.0           |              |

5 rows × 149 columns

Toda e qualquer transformação aplicada aos dados de treino, deve ser aplicada aos dados de teste e aos novos dados.

```
In [97]: # Removendo as colunas categóricas originais e adicionando as codificadas nos dados de Teste
```

```
X_test_preprocessed = X_test.drop(categorical_cols, axis = 1).reset_index(drop = True)
X_test_preprocessed = pd.concat([X_test_preprocessed, X_test_encoded], axis = 1)
```

```
In [98]: # Visualizando as primeiras linhas dos dados de teste após o pré-processamento
X_test_preprocessed.head()
```

```
Out[98]:
```

|   | SALARIO | VALOR_TICKET-CESTA | SOMA_APOSENTADORIAS+PENSAO | D0023 | SOMA_BOLSAS_FAMILIA | SOMA_BPC-LOAS | SOMA_SEGUROS |
|---|---------|--------------------|----------------------------|-------|---------------------|---------------|--------------|
| 0 | 0.0     | 0.0                | 0.0                        | 0.0   | 0.0                 | 0.0           | 0.0          |
| 1 | 0.0     | 0.0                | 0.0                        | 0.0   | 0.0                 | 0.0           | 0.0          |
| 2 | 0.0     | 0.0                | 4000.0                     | 0.0   | 0.0                 | 0.0           | 0.0          |
| 3 | 0.0     | 0.0                | 2100.0                     | 0.0   | 0.0                 | 0.0           | 0.0          |
| 4 | 2400.0  | 0.0                | 0.0                        | 0.0   | 0.0                 | 0.0           | 0.0          |

5 rows × 149 columns

• Padronizando os dados para que estejam na mesma escala, usando o *StandardScaler*:

( Não é necessário no *RandomForest*, mas faremos para aprender )

```
In [99]: # Criando uma lista pandas para receber os nomes das colunas com valores numéricos (int64 e float64), excluindo as demais
numeric_cols = X_train_preprocessed.select_dtypes(include = ['int64', 'float64']).columns

print(numeric_cols)
```

```
Index(['SALARIO', 'VALOR_TICKET-CESTA', 'SOMA_APOSENTADORIAS+PENSAO', 'D0023',
      'SOMA_BOLSAS_FAMILIA', 'SOMA_BPC-LOAS', 'SOMA_SEGUROS_DESEMPREGO',
      'D0073', 'UF_AC', 'UF_AL',
      ...,
      'C007D_99.0', 'C010_1.0', 'C010_99.0', 'SOMENTE_BENEFICIO_N/A',
      'SOMENTE_BENEFICIO_Sim', 'NAO_REMUNERADO_N/A', 'NAO_REMUNERADO_Sim',
      'C014_1.0', 'C014_2.0', 'C014_99.0'],
      dtype='object', length=149)
```

```
In [100]: # Criando o StandardScaler

scaler = StandardScaler()

# Obs: Está sendo criado o objeto 'scaler', como uma instância da classe StandardScaler() do sklearn
# Ele servirá p/ deixar todas as colunas numéricas com média 0 e desvio padrão = 1
```

```
In [101]: # Aplicando o StandardScaler às variáveis numéricas

X_train_preprocessed[numeric_cols] = scaler.fit_transform(X_train_preprocessed[numeric_cols])
X_test_preprocessed[numeric_cols] = scaler.transform(X_test_preprocessed[numeric_cols])
```

```
In [102]: # Visualizando as primeiras linhas dos datasets de treino e teste após pré-processamento
X_train_preprocessed.head()
```

```
Out[102]:
```

|   | SALARIO   | VALOR_TICKET-CESTA | SOMA_APOSENTADORIAS+PENSAO | D0023     | SOMA_BOLSAS_FAMILIA | SOMA_BPC-LOAS | SOMA_SEGUROS |
|---|-----------|--------------------|----------------------------|-----------|---------------------|---------------|--------------|
| 0 | 0.106581  | -0.032049          | 0.143776                   | -0.129761 | -0.270754           | -0.179414     | 0.0          |
| 1 | -0.386797 | -0.032049          | 0.143776                   | -0.129761 | -0.270754           | -0.179414     | 0.0          |
| 2 | 0.132548  | -0.032049          | -0.432151                  | -0.129761 | -0.270754           | 4.805993      | 0.0          |
| 3 | -0.386797 | -0.032049          | -0.432151                  | -0.129761 | 3.165902            | -0.179414     | 0.0          |
| 4 | -0.009092 | -0.032049          | 0.719703                   | -0.129761 | -0.270754           | -0.179414     | 0.0          |

5 rows × 149 columns

Toda e qualquer transformação aplicada aos dados de treino, deve ser aplicada aos dados de teste e aos novos dados (nestes 2 últimos, sem o *fit*)

```
In [103]: X_test_preprocessed.head()
```

```
Out[103]:
```

|   | SALARIO   | VALOR_TICKET-CESTA | SOMA_APOSENTADORIAS+PENSAO | D0023     | SOMA_BOLSAS_FAMILIA | SOMA_BPC-LOAS | SOMA_SEGUR |
|---|-----------|--------------------|----------------------------|-----------|---------------------|---------------|------------|
| 0 | -0.386797 | -0.032049          | -0.432151                  | -0.129761 | -0.270754           | -0.179414     |            |
| 1 | -0.386797 | -0.032049          | -0.432151                  | -0.129761 | -0.270754           | -0.179414     |            |
| 2 | -0.386797 | -0.032049          | 1.772355                   | -0.129761 | -0.270754           | -0.179414     |            |
| 3 | -0.386797 | -0.032049          | 0.725215                   | -0.129761 | -0.270754           | -0.179414     |            |
| 4 | 0.746319  | -0.032049          | -0.432151                  | -0.129761 | -0.270754           | -0.179414     |            |

5 rows × 149 columns

## 8) Instanciando/ Criando o modelo:

```
In [104]: # Criando o modelo RandomForest
dsa_modelo_v1 = RandomForestClassifier(random_state = 42)
```

## 9) Treinando o modelo:

```
In [105]: # Treinando o modelo
dsa_modelo_v1.fit(X_train_preprocessed, y_train)

# OBS: ESTE TREINAMENTO DEMOROU 2 MIN P/ SER FEITO EM MEU LENOVO CORE i5 2.40 GHz, 8Gb RAM e c/ SSD
```

```
Out[105]: RandomForestClassifier(random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.  
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

## 10) Avaliando/ Visualizando o modelo:

### • Gráfico da "Taxa de Erro" da Floresta Aleatória:

```
In [106]: # plotando um gráfico para mostrar a taxa de erro do modelo em função do nro de árvores de decisão

#n_trees = [10, 50, 100, 200, 300, 400, 500]
#error_rates = []
#
#for n in n_trees:
#    rf = RandomForestClassifier(n_estimators=n, random_state=42)
#    rf.fit(X_train_preprocessed, y_train)
#    y_pred = rf.predict(X_test_preprocessed)
#    error = 1 - accuracy_score(y_test, y_pred)
#    error_rates.append(error)
#
#plt.figure(figsize=(8,5))
#plt.plot(n_trees, error_rates, marker='o')
#plt.title('Taxa de Erro vs Número de Árvores (Random Forest)')
#plt.xlabel('Número de Árvores (n_estimators)')
#plt.ylabel('Taxa de Erro')
#plt.grid(True)
#plt.show()

# OBS: ESTE GRÁFICO DEMOROU 40 MIN P/ SER FEITO EM MEU LENOVO CORE i5 2.40 GHz, 8Gb RAM e c/ SSD
```

Como podemos observar, o modelo tem a MENOR taxa de erro EM TORNO DE 400 árvores de decisão. A partir daí, o modelo começa a errar mais.

• Gráfico da "Importância de Cada Variável":

```
In [107]: # Descobrendo quais são as variáveis mais importantes do modelo
importances = dsa_modelo_v1.feature_importances_
feature_names = X_train_preprocessed.columns

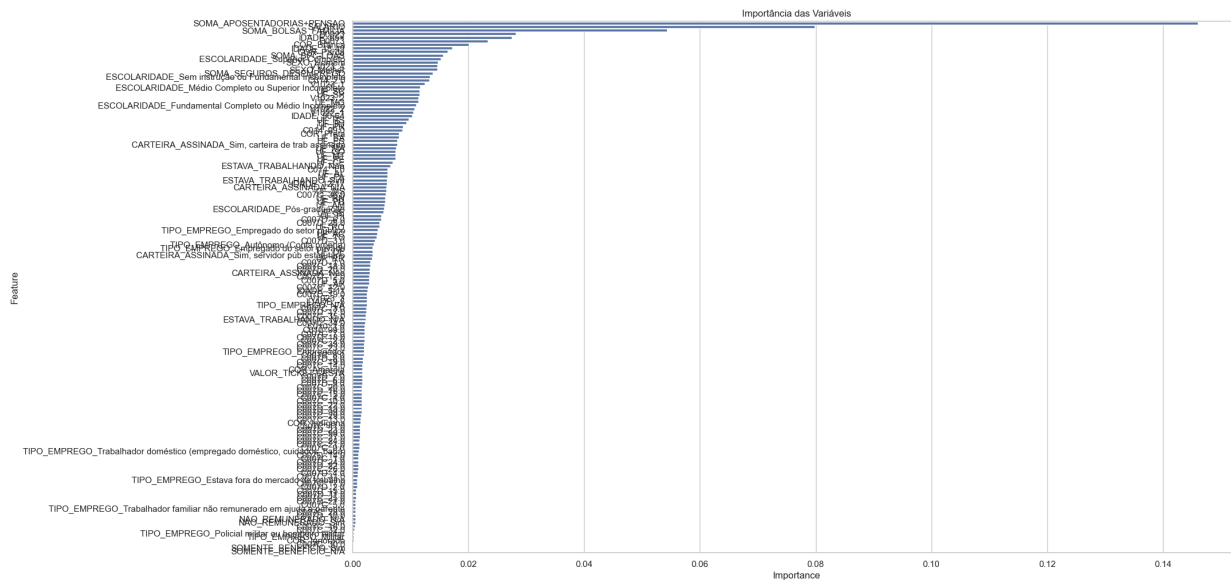
# Criando um DataFrame para armazenar as importâncias
importances_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
importances_df = importances_df.sort_values(by='Importance', ascending=False)
importances_df
```

```
Out[107]:
```

|     | Feature                    | Importance |
|-----|----------------------------|------------|
| 2   | SOMA_APOSENTADORIAS+PENSAO | 0.146075   |
| 0   | SALARIO                    | 0.079866   |
| 4   | SOMA_BOLSAS_FAMILIA        | 0.054322   |
| 3   | D0023                      | 0.028237   |
| 46  | IDADE_65+                  | 0.027523   |
| ... | ...                        | ...        |
| 68  | TIPO_EMPREGO_Militar       | 0.000148   |
| 51  | COR_Ignorado               | 0.000139   |
| 106 | C007C_30.0                 | 0.000062   |
| 143 | SOMENTE_BENEFICIO_Sim      | 0.000061   |
| 142 | SOMENTE_BENEFICIO_N/A      | 0.000060   |

149 rows × 2 columns

```
In [108]: # Plotando o gráfico de importância das variáveis
plt.figure(figsize=(20, 12))
sns.barplot(data=importances_df, x='Importance', y='Feature')
plt.title('Importância das Variáveis')
plt.show()
```



• Fazendo previsões com os dados de TESTE:

```
In [109]: # Fazendo previsões no conjunto de teste (retornará um array numpy)

y_pred = dsa_modelo_v1.predict(X_test_preprocessed)
y_pred
```

```
Out[109]: array([2, 1, 2, ..., 2, 1, 2])
```

```
In [110]: # Mesmo array anterior, mas contendo as probabilidades das previsões
```

```
y_pred_proba = dsa_modelo_v1.predict_proba(X_test_preprocessed)[: ,1]
y_pred_proba
```

```
Out[110]: array([0.59694855, 0.        , 1.        , ..., 0.91        , 0.46860748,
                0.66725    ])
```

```
In [111]: # tabela com a contagem das previsões iguais a 1 (AUX. EMERGENCIAL) e iguais a 2 (SEM AUXÍLIO)
```

```
pd.Series(y_pred).value_counts()
```

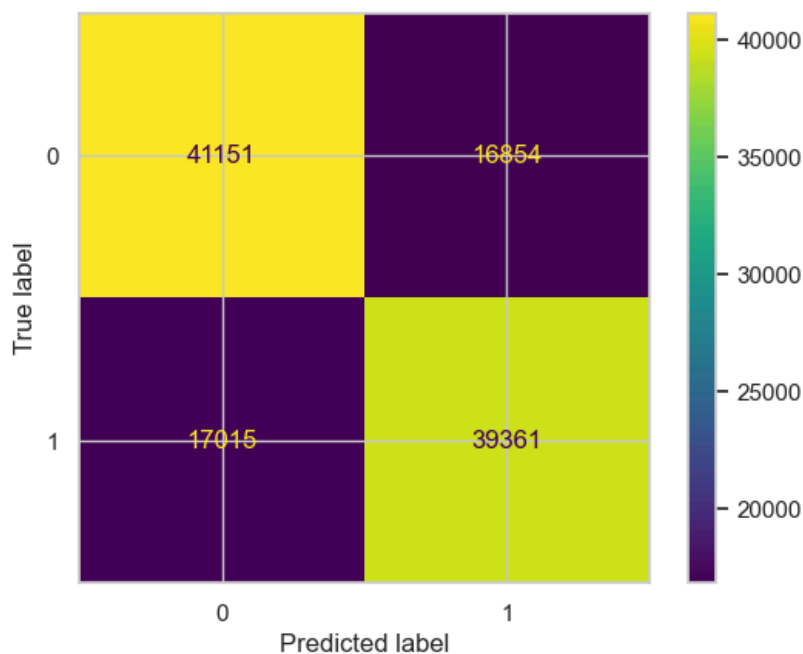
```
Out[111]: 1    58166
          2    56215
          Name: count, dtype: int64
```

#### • Criando a Matriz de Confusão:

```
In [112]: # Matriz de confusão
```

```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

```
Out[112]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2333f79c580>
```



#### • Avaliação das métricas do modelo:

```
In [113]: # Avaliando o modelo pela acurácia (% Acerto, i.e.: Acertos /Total previsões). Usamos ela p/ comparar modelos
```

```
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

```
# A Acurácia é uma métrica global, ou seja, para todo o modelo, e é especialmente útil
# quando as classes estão balanceadas
```

```
Out[113]: 0.7038931291036099
```

```
In [114]: # Avaliando o modelo pela AUC (Área Sob a Curva ROC). Usamos p/ comparar diferentes algoritmos!
```

```
auc = roc_auc_score(y_test, y_pred_proba)
auc
```

```
# AUC também é uma métrica global, e mede a capacidade de separação entre as classes.
# Ela é especialmente útil em dados desbalanceados.
```

```
Out[114]: 0.7734955629109497
```

**Acurácia e AUC são MÉTRICAS GLOBAIS**, ou seja, para todo o modelo! O ideal é que estejam acima de 70%

```
In [115]: # Outras métricas importantes p/ classificação com RF
```

```
classification_rep = classification_report(y_test, y_pred)
print(classification_rep)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 1            | 0.71      | 0.71   | 0.71     | 58005   |
| 2            | 0.70      | 0.70   | 0.70     | 56376   |
| accuracy     |           |        | 0.70     | 114381  |
| macro avg    | 0.70      | 0.70   | 0.70     | 114381  |
| weighted avg | 0.70      | 0.70   | 0.70     | 114381  |

**Métricas Locais** (por classes). Pela tabela acima, podemos perceber que a **Precisão**, o **Recall** e o **F1-Score** são praticamente os mesmos (70%) em ambas as classes (1: com auxílio emergencial; e 2: sem auxílio emergencial), o que comprova que o modelo está equilibrado.

#### • Validação Cruzada para checar a performance em cada fold:

```
In [116]: # Criando um novo modelo RandomForest para validação cruzada
```

```
#dsa_modelo_cv = RandomForestClassifier(random_state = 42)
```

```
In [117]: # Realizando a validação cruzada
```

```
# Usaremos 5 folds para a validação cruzada
```

```
#cv_scores = cross_val_score(dsa_modelo_cv, X_train_preprocessed, y_train, cv = 5)
```

```
#cv_scores
```

```
# OBS: ESTA ANÁLISE DEMOROU 8 MIN P/ SER FEITO EM MEU LENOVO CORE i5 2.40 GHz, 8Gb RAM e c/ SSD
```

**CONCLUSÃO:** Como a performance em cada fold **NÃO VARIA** muito, quer dizer que **EXISTE UM PADRÃO** que o modelo conseguiu capturar nos dados - "**Em diferentes amostras (folds), o modelo é consistente!**". Estes resultados fornecem uma visão mais robusta do desempenho do modelo, pois a validação cruzada avalia a capacidade do modelo de generalizar para novos dados. A variação nas pontuações de acurácia entre os diferentes folds indicaria que o modelo poderia vir a se comportar de maneira inconsistente em diferentes subconjuntos dos dados. Isso pode ser devido a características dos dados, como desbalanceamento de classes, ou à necessidade de um ajuste mais fino dos hiperparâmetros do modelo.

## 11) Otimização de Hiperparâmetros e Versão Final do Modelo:

#### • Otimização de Hiperparâmetros:

In [118]: *# Definindo os hiperparâmetros para serem otimizados (dicionário python)*

```
param_grid = {
    'n_estimators': [50, 100, 200], # Número de árvores
    'max_depth': [None, 10, 20, 30], # Profundidade máxima da árvore
    'min_samples_split': [2, 4, 6], # Número mínimo de amostras para dividir um nó
    'min_samples_leaf': [1, 2, 4] # Número mínimo de amostras exigido em um nó folha
}
```

In [119]: *# Criando o modelo RandomForest para a otimização*

```
dsa_modelo_opt = RandomForestClassifier(random_state = 42)
```

In [120]: *# Configurando a busca em grade com validação cruzada*

```
grid_search = GridSearchCV(dsa_modelo_opt, param_grid, cv = 5, scoring = 'accuracy', n_jobs = -1)
```

```
# O objeto "grid_search" criado nesta célula, é uma instância da classe GridSearchCV().
# Ele foi configurado para buscar os melhores hiperparâmetros do modelo "dsa_modelo_opt"
# usando o dicionário "param_grid", criado na célula acima, e validação cruzada com 5 folds.
```

```
# Ele fará um cruzamento para combinar todos os possíveis valores dos hiperparâmetros escolhidos
# e guardará os melhores valores na variável "best_params_" (atributo do obj "grid_search")
```

In [121]: *# Realizando a otimização com o conjunto de treino*

```
# ATENÇÃO! ESSA BUSCA PELOS MELHORES HIPERPARÂMETROS PODE SER BASTANTE DEMORADA
# A DEPENDER DO NRO DE VARIÁVEIS ENVOLVIDAS E DO NRO DE HIPERPARÂMETROS USADOS
# TENHA MUUUUITA PACIÊNCIAAAAAAAAAAAAA
```

```
#grid_search.fit(X_train_preprocessed, y_train)
```

```
# PS:
```

```
# Dimensões X_train_preprocessed (shape): 266.889 x 149 (70% dos dados originais p/ TREINO)
```

```
# Número de árvores (n_estimators): [50, 100, 200]
```

```
# Profundidade máxima da árvore (max_depth): [None, 10, 20, 30]
```

```
# Número mínimo de amostras para dividir um nó (min_samples_split): [2, 4, 6]
```

```
# Número mínimo de amostras exigido em um nó folha (min_samples_leaf): [1, 2, 4]
```

```
# OBS: ESTA OTIMIZAÇÃO DEMOROU 285 MIN (4:45h) P/ SER FEITA EM MEU LENOVO CORE i5 2.40 GHz, 8Gb RAM e c/ S
```

In [122]: *# Melhores parâmetros e pontuação*

```
#best_params = grid_search.best_params_
#best_score = grid_search.best_score_
```

In [123]: *#best\_params, best\_score*

**PS:** A título de curiosidade, os melhores hiperparâmetros encontrados durante a otimização nesse conjunto de dados foram:

**max\_depth:** 30

**min\_samples\_leaf:** 2

**min\_samples\_split:** 2

**n\_estimators:** 200

(Deixamos comentadas estas células pois essa otimização foi muito demorada para ser feita!)

• **Versão Final do Modelo:**

```
In [124]: # Criando o modelo RandomForest com os melhores hiperparâmetros
```

```
#dsa_modelo_final = RandomForestClassifier(n_estimators = best_params['n_estimators'],  
#                                         max_depth = best_params['max_depth'],  
#                                         min_samples_split = best_params['min_samples_split'],  
#                                         min_samples_leaf = best_params['min_samples_leaf'],  
#                                         random_state = 42)
```

```
In [125]: # Treinando o modelo final com o conjunto de treino
```

```
#dsa_modelo_final.fit(X_train_preprocessed, y_train)  
  
# OBS: ESTE TREINAMENTO DEMOROU 2 MIN P/ SER FEITO EM MEU LENOVO CORE i5 2.40 GHz, 8Gb RAM e c/ SSD
```

```
In [126]: # Previsões com dados de teste
```

```
#y_pred_final = dsa_modelo_final.predict(X_test_preprocessed)  
#y_pred_final
```

```
In [127]: # Mesmo array anterior, mas contendo as probabilidades das previsões
```

```
#y_pred_proba_final = dsa_modelo_final.predict_proba(X_test_preprocessed)[: ,1]  
#y_pred_proba_final
```

```
In [128]: # tabela com a contagem das previsões iguais a 1 e 2
```

```
#pd.Series(y_pred_final).value_counts()
```

```
In [129]: # Avaliando a acurácia do modelo final no conjunto de teste
```

```
#final_accuracy = accuracy_score(y_test, y_pred_final)  
#final_accuracy
```

```
In [130]: # Avaliando o modelo final pela AUC (Área Sob a Curva ROC)
```

```
#final_auc = roc_auc_score(y_test, y_pred_proba_final)  
#final_auc
```

```
In [131]: #final_classification_report = classification_report(y_test, y_pred_final)
```

```
#print(final_classification_report)
```

Com a Otimização dos Hiperparâmetros, conseguimos melhorar a **Acurácia** do modelo em **1,89% (nova acurácia: 72,27)** e melhoramos a **AUC** em **2,62% (nova AUC: 79,97)**.

Mas, como o tpo de execução desta Otimização de Hiperparâmetros ficou longo e o ganho percentual foi baixo, optamos por deixar comentadas as linhas desta Seção 11 (Otimização), e vamos seguir adiante com o modelo **dsa\_modelo\_v1** com **70,39% de acurácia** e **77,35% de AUC**!

**PS:** Se quiser, crie uma **versão 2** do modelo com os hiperparâmetros que havíamos encontrado:

max\_depth: 30

min\_samples\_leaf: 2

min\_samples\_split: 2

n\_estimators: 200

## ->SALVAMENTO/ DEPLOY DO MODELO:

```
In [132]: #joblib.dump(dsa_modelo_final, 'dsa_modelo_final.pkl') #Não vamos optar pela Otimização de Hiperparâmetros
joblib.dump(dsa_modelo_v1, 'dsa_modelo_v1.pkl')
```

```
Out[132]: ['dsa_modelo_v1.pkl']
```

```
In [133]: joblib.dump(scaler, 'dsa_padronizador.pkl')
```

```
Out[133]: ['dsa_padronizador.pkl']
```

```
In [134]: print(X_train_preprocessed.shape)
X_train_preprocessed.head()
```

(266889, 149)

```
Out[134]:
```

|   | SALARIO   | VALOR_TICKET-CESTA | SOMA_APOSENTADORIAS+PENSAO | D0023     | SOMA_BOLSAS_FAMILIA | SOMA_BPC-LOAS | SOMA_SEGURO |
|---|-----------|--------------------|----------------------------|-----------|---------------------|---------------|-------------|
| 0 | 0.106581  | -0.032049          | 0.143776                   | -0.129761 | -0.270754           | -0.179414     |             |
| 1 | -0.386797 | -0.032049          | 0.143776                   | -0.129761 | -0.270754           | -0.179414     |             |
| 2 | 0.132548  | -0.032049          | -0.432151                  | -0.129761 | -0.270754           | 4.805993      |             |
| 3 | -0.386797 | -0.032049          | -0.432151                  | -0.129761 | 3.165902            | -0.179414     |             |
| 4 | -0.009092 | -0.032049          | 0.719703                   | -0.129761 | -0.270754           | -0.179414     |             |

5 rows × 149 columns

## ->CARREGANDO E TESTANDO O MODELO/PADRONIZADOR SALVOS:

```
In [135]: # Carregando o modelo e o padronizador (scaler)
```

```
modelo_carregado = joblib.load('dsa_modelo_v1.pkl')
scaler_carregado = joblib.load('dsa_padronizador.pkl')
```

```
In [136]: # Exportando (CSV) a tabela q havia sido criada c/ o One-Hot Encoding antes da padronização (para simular
# (mesmo código da célula 95 deste JupyterNobebook)
```

```
X_train_preprocessed = X_train.drop(categorical_cols, axis = 1).reset_index(drop = True)
X_train_preprocessed = pd.concat([X_train_preprocessed, X_train_encoded], axis = 1)
X_train_preprocessed.to_csv('NOVOS_DADOS_N_PADRONIZADOS.csv', index = False, encoding = 'utf-8')
```

```
In [137]: # Carregando os novos dados em um DataFrame do Pandas
```

```
df_novos_dados = pd.read_csv("NOVOS_DADOS_N_PADRONIZADOS.csv")
print(df_novos_dados.shape)
```

(266889, 149)

```
In [138]: # Padronizando os novos dados com o scaler (padronizador) q acabamos de carregar na memória ('scaler_carregado')
# e fazendo o 'deploy' (previsões) via linha de comando
```

```
novos_dados_padronizados = scaler_carregado.transform(df_novos_dados)
y_pred = modelo_carregado.predict(novos_dados_padronizados)
print(y_pred)
```

[1 1 2 ... 2 1 1]

```
In [139]: %watermark -a "Christopherson B. Francischetti"
```

Author: Christopherson B. Francischetti

```
In [140]: %watermark -v -m
```

```
Python implementation: CPython
Python version       : 3.9.13
IPython version      : 7.31.1

Compiler            : MSC v.1916 64 bit (AMD64)
OS                  : Windows
Release             : 10
Machine             : AMD64
Processor           : Intel64 Family 6 Model 140 Stepping 1, GenuineIntel
CPU cores           : 8
Architecture        : 64bit
```

```
In [141]: %watermark --iversions
```

```
sklearn   : 1.3.2
seaborn    : 0.13.1
matplotlib: 3.8.2
pandas     : 2.1.4
joblib     : 1.3.2
numpy      : 1.26.3
```

**Fim**